universität koblenz
weiter:denken
Faculty 4: Computer Science

WeST
Web Science and Technologies
Institute for Web Science
and Technologies

Benchmarking Holistic vs Sentence-Level Reward-Model Preferences for Factual Summarization via Granular Credit Assignment (GCA)

Master's Thesis

in partial fulfillment of the requirements for
the degree of Master of Science (M.Sc.)
in Web and Data Science

submitted by
Muhammad Hasnat

(Matriculation Number: 221100612)

First supervisor: Prof. Dr. Frank Hopfgartner
Institute for Web Science and Technologies

Second supervisor: Dr. Zeyd Boukhers
Fraunhofer Institute for Applied Information Technology FIT

Koblenz, July 2026

Statement

I hereby certify that this thesis has been composed by me and is based on my own work, that I did not use any further resources than specified – in particular no references unmentioned in the reference section – and that I did not submit this thesis to another examination before. The paper submission is identical to the submitted electronic version.

	Yes	No
I agree to have this thesis published in the library.	☐	☐
I agree to have this thesis published on the Web.	☐	☐
The thesis text is available under a Creative Commons License (CC BY-SA 4.0).	☐	☐
The source code is available under a GNU General Public License (GPLv3).	☐	☐
The collected data is available under a Creative Commons License (CC BY-SA 4.0).	☐	☐

.....

(Place, Date) (Signature)

Note

- If you would like us to contact you for the graduation ceremony, please provide your personal E-mail address: .....
- If you would like us to send you an invite to join the WeST Alumni and Members group on LinkedIn, please provide your LinkedIn ID : .....

Zusammenfassung

Die präferenzbasierte Ausrichtung abstraktiver Zusammenfassungsmodelle stützt sich zunehmend auf automatisch erzeugte Präferenzlabels anstelle menschlicher Bewertungen. Unklar bleibt, ob ein einzelnes ganzheitliches Urteil über eine mehrsätzige Zusammenfassung das verfügbare Supervisionssignal ebenso wirksam nutzt wie ein Urteil, das aus satzweiser Evidenz zusammengesetzt wird. Diese Arbeit untersucht diese Frage als kontrollierten Vergleich zweier Verfahren der Präferenzkonstruktion. In der ganzheitlichen Bedingung wird eine vollständige Kandidatenzusammenfassung gegen den Quellartikel bewertet, und aus den Bewertungen wird eine paarweise Präferenz gebildet. In der Bedingung Granular Credit Assignment (GCA) wird jede Zusammenfassung in Sätze zerlegt, jeder Satz einzeln bewertet, und die Satzbewertungen werden zu einer Präferenz auf Zusammenfassungsebene aggregiert. Die Kandidatenzusammenfassungen, das Bewertungsmodell (AlignScore), die Bradley–Terry-Architektur des Belohnungsmodells sowie das Trainingsverfahren bleiben unverändert, sodass die Granularität des Feedbacks die zentrale experimentelle Variable bildet.

Die abhängige Variable ist die Erlernbarkeit der Präferenzen, gemessen als paarweise Validierungsgenauigkeit eines auf den jeweiligen Präferenzpaaren trainierten Belohnungsmodells. Über sechs wiederholte Läufe bei $n = 1.000$ ergibt sich für GCA-Präferenzen ein durchschnittlicher Vorteil von etwa 3,08 Prozentpunkten gegenüber der ganzheitlichen Präferenzkonstruktion, wobei GCA in fünf von sechs Läufen vorne liegt. Unsicherheitsschätzungen auf Basis von Kreuzvalidierungs-Folds sind zurückhaltend zu interpretieren, da Folds innerhalb eines Laufs nicht unabhängig sind; auf Laufebene erreicht die Evidenz das konventionelle Niveau von 0,05 nicht. Der Vorteil bleibt bei größerem Datenumfang nicht in gleicher Höhe bestehen: Er beträgt etwa $-0{,}42$ Prozentpunkte bei $n = 5.000$ und $+0{,}35$ Prozentpunkte bei $n = 10.000$, während die absolute Genauigkeit in beiden Bedingungen zunimmt. Ein explorativer Vergleich der Bewertungsmodi legt nahe, dass eine externe satzweise Zerlegung vor allem dann nützlich sein könnte, wenn das Bewertungsmodell nicht bereits intern eine vergleichbare Zerlegung vornimmt.

Die Ergebnisse charakterisieren, unter welchen Bedingungen granulares Faktizitäts-Feedback die Struktur eines Präferenzlernsignals verändert, und belegen nicht, dass GCA allgemein vorzuziehen ist. Eine höhere Genauigkeit des Belohnungsmodells zeigt an, dass eine Präferenzrelation von der gewählten Architektur konsistenter rekonstruiert werden kann. Sie belegt für sich genommen weder, dass GCA-Labels besser mit menschlichen Faktizitätsurteilen übereinstimmen, noch dass auf solchen Präferenzen trainierte Zusammenfassungsmodelle faktisch konsistentere Texte erzeugen; beide Fragen bleiben künftiger Arbeit vorbehalten.

Abstract

Preference-based alignment of abstractive summarization models increasingly relies on automatically generated preference labels rather than human judgments. It remains unclear whether a single holistic judgment over a multi-sentence summary uses the available supervision as effectively as a judgment assembled from sentence-level evidence. This thesis studies that question as a controlled comparison of two preference-construction procedures. In the holistic condition, a complete candidate summary is scored against the source article and the resulting scores form a pairwise preference. In the Granular Credit Assignment (GCA) condition, each summary is decomposed into sentences, each sentence is scored separately, and the sentence scores are aggregated into a summary-level preference. The candidate summaries, the factuality evaluator (AlignScore), the Bradley–Terry reward-model architecture, and the training procedure are held fixed, so that feedback granularity is the primary experimental variable.

The dependent variable is preference learnability, measured as the pairwise validation accuracy of a reward model trained on the resulting preference pairs. Across six repeated runs at $n = 1{,}000$, GCA achieved an average advantage of approximately 3.08 percentage points over holistic preference construction, favouring GCA in five of six runs. Uncertainty estimates based on cross-validation folds should be interpreted cautiously, because folds within a run are not independent; at the run level the evidence does not meet the conventional 0.05 threshold. The advantage did not remain equally large at larger sample sizes: it is approximately -0.42 percentage points at $n = 5{,}000$ and $+0.35$ percentage points at $n = 10{,}000$, while absolute accuracy improves for both conditions. An exploratory comparison of evaluator modes suggests that external sentence-level decomposition may be most useful when the evaluator does not already perform comparable decomposition internally.

These results characterize when granular factuality feedback changes the structure of a preference-learning signal, rather than establishing that GCA is generally preferable. Higher reward-model accuracy indicates that a preference relation is more consistently recoverable by the chosen architecture. It does not by itself establish that GCA labels agree better with human factuality judgments, nor that summarization models trained on such preferences produce more factually consistent summaries; both questions are left to future work.

Acknowledgments

I would like to thank my first supervisor, Prof. Dr. Frank Hopfgartner of the Institute for Web Science and Technologies, for his guidance throughout this project and in particular for the feedback that led me to narrow the study to a single, cleanly controlled comparison rather than a broader but less conclusive pipeline. I am equally grateful to my second supervisor, Dr. Zeyd Boukhers of the Fraunhofer Institute for Applied Information Technology FIT, for his technical input on reward modelling and factuality evaluation.

The experiments in this thesis were carried out on the MOGON NHR cluster at Johannes Gutenberg University Mainz. I gratefully acknowledge the compute time made available there; the repeated-run and multi-scale campaigns reported in Chapter 6 would not have been feasible without access to that infrastructure.

Finally, I thank the University of Koblenz for providing the research environment in which this work was completed, and my family and friends for their support over the course of it.

Declaration on the Use of AI Assistance

I acknowledge the use of large language models to support the writing of this thesis, specifically for improving the clarity, structure, and grammatical correctness of the text, and as a coding assistant during the implementation of the experimental pipeline. The research design, the experiments, the analysis of the results, and the conclusions drawn from them are my own. All generated text and code were reviewed, verified, and revised by me, and I take full responsibility for the content of this thesis.

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Full Term
AI	Artificial Intelligence
CI	Confidence Interval
CV	Cross-Validation
DPO	Direct Preference Optimization
GCA	Granular Credit Assignment
GPU	Graphics Processing Unit
LLM	Large Language Model
MoDPO	Multi-Objective Direct Preference Optimization
NHR	Nationales Hochleistungsrechnen (National High-Performance Computing)
NLI	Natural Language Inference
RLAIF	Reinforcement Learning from AI Feedback
RLHF	Reinforcement Learning from Human Feedback
RM	Reward Model
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
`_sp`	Suffix marking the AlignScore evaluation modes that split the input internally (`nli_sp`, `bin_sp`)

1. Introduction

1.1. Motivation

Large language models are increasingly used for multi-sentence generation tasks such as news summarization. In this setting, the perceived quality of an output is often dominated not by its overall fluency but by a small number of high-impact factual errors: a wrong entity, a flipped relation, or a fabricated claim can outweigh otherwise well-written text. Throughout this thesis, *reliability* refers specifically to factual consistency with the source document, not to stylistic polish or fluency.

Preference-based fine-tuning, most prominently reinforcement learning from human feedback (RLHF), updates a model using comparative judgments between candidate outputs rather than token-level supervision [3, 15, 21]. Collecting high-quality human preferences at scale is expensive and slow, so reinforcement learning from AI feedback (RLAIF) replaces human annotators with an automatic judge [1, 12]. This trades annotation cost for a new source of label noise and bias.

A further question arises for long, multi-sentence outputs such as news summaries, one that is largely independent of whether the feedback comes from a human or an AI judge: how granular should the preference signal be? A single holistic judgment ("summary A is better than summary B") collapses all of the evidence in a multi-sentence output into one binary decision. If a summary is mostly accurate but contains one fabricated sentence, a holistic judge may still prefer it over an alternative that is uniformly mediocre but contains no outright fabrication. The holistic label cannot express that distinction. Sentence-level judging can in principle localize exactly which claims are, and are not, supported by the source, and that localized evidence can then be aggregated into a summary-level decision. This thesis asks whether that extra granularity actually produces a better training signal, and under what conditions.

1.2. Problem Statement

When constructing AI-generated preference data for training a factuality-oriented reward model, it is unclear whether coarse, holistic preferences are sufficient, or whether they systematically underuse the supervision signal available in long, multi-sentence outputs. The problem addressed here, under controlled conditions, is whether sentence-level factuality judgments, aggregated into a summary-level preference through a mechanism referred to as *Granular Credit Assignment* (GCA), yield

a more learnable preference signal for reward-model training than purely holistic judgments of the same candidate summaries.

"More learnable" is given a concrete, testable meaning throughout: the pairwise validation accuracy of a Bradley–Terry reward model trained on the resulting preference pairs (Chapter 3, Section 3.1). Given a held-out preference pair, how often does the trained reward model rank the summary the judge preferred above the alternative?

1.3. Thesis Objective

The objective of this thesis is to determine, through a controlled empirical comparison, whether Granular Credit Assignment produces a more learnable factuality-preference signal than holistic scoring. The two conditions differ only in how factuality scores are obtained and converted into a pairwise label:

> **Holistic:** article + complete candidate summary → factuality score → pairwise preference.
>
> **GCA:** article + individual sentences → sentence factuality scores → aggregation → pairwise preference.

Both preference sets are then used to train the same Bradley–Terry reward-model architecture under the same procedure. Holding the source articles, the candidate summaries and their generation settings, the evaluator family, the reward-model architecture, and the training procedure fixed allows feedback granularity to be studied as the primary experimental variable, rather than as one difference among several.

The thesis does not propose a new reward-model architecture, a new factuality metric, or a new training objective. Its subject is the construction of the preference data itself.

1.4. Research Questions

Three research questions follow from this objective. They are stated here in summary form; Chapter 4, Section 4.1 gives them together with the working hypotheses and the scope conditions under which each can be answered, and Chapter 8, Section 8.2 answers them against the evidence collected.

- **RQ1.** Does sentence-level factuality feedback aggregated via GCA produce a more learnable pairwise preference signal than holistic summary-level feedback, as measured by reward-model pairwise validation accuracy?
- **RQ2.** How do evaluator configuration and sentence-score aggregation strategy affect the relative learnability of holistic and GCA-derived preferences?

- **RQ3.** How stable are the observed differences across repeated reward-model training runs and dataset sizes?

RQ1 is the confirmatory question and is addressed by a repeated-run comparison. RQ2 is addressed by an exploratory sweep, and therefore supports statements about association between configuration and outcome rather than about causal mechanism. RQ3 motivates both the repeated runs at a single dataset size and the multi-scale check at $n = 5{,}000$ and $n = 10{,}000$.

1.5. Contributions

This work makes the following contributions:

1. A controlled framework for comparing holistic and sentence-level, GCA-based construction of factuality preferences for abstractive summarization, in which the candidate pool, evaluator, reward-model architecture, and training procedure are held fixed (Chapter 4).

2. An empirical study of how evaluator mode and aggregation strategy interact with feedback granularity. Neither the aggregation formula nor the evaluator configuration is neutral: a penalty-weighted aggregation performs worse than a simple mean, and in an exploratory sweep the evaluator mode is associated with whether an advantage for GCA is observed at all (Chapter 6, Sections 6.3 and 6.5).

3. A repeated-run and multi-scale evaluation showing that GCA can improve preference learnability under some configurations, most clearly at $n = 1{,}000$ with the AlignScore `nli` mode, and that the improvement diminishes at larger dataset sizes (Sections 6.6 and 6.7).

4. An exploratory observation that the benefit of external sentence-level decomposition may be reduced when the underlying evaluator already performs comparable internal decomposition, offered as a hypothesis motivated by the mode comparison rather than as an established mechanism (Chapter 7, Section 7.2).

5. A transparent account of instability, stochastic variation across runs, methodological limitations, and the reproducibility of the pipeline (Chapters 5, 7, and 8).

These contributions concern the construction and learnability of preference data. The thesis does not claim that GCA yields objectively more accurate factuality labels, nor that it improves the factual consistency of generated summaries; the scope of what the experiments support is stated precisely in Chapter 4, Section 4.3.

1.6. Scope

- **Task:** single-document abstractive news summarization, using the CNN/DailyMail dataset [8, 14].
- **Candidate generation:** a single instruction-tuned base model, Mistral-7B-Instruct-v0.3, sampled at two temperatures ($T = 0.7$ and $T = 1.0$) to produce an A/B candidate pair per source article.
- **Judge:** a single fixed, deterministic factuality judge, `yzha/AlignScore` [23], in its `nli` evaluation mode. No generative LLM-as-judge and no external API dependency is used in the final pipeline.
- **Optimization target:** the learnability of the resulting preference data for Bradley–Terry reward-model training, measured by pairwise validation accuracy under 5-fold cross-validation. A downstream fine-tuned generation policy is not evaluated in the final comparison; an early Direct Preference Optimization (DPO) prototype was explored but dropped from the final scope (Chapter 4).

1.7. Thesis Structure

Chapter 2 situates this work within the literature on preference-based alignment, LLM-as-a-judge reliability, fine-grained feedback for long-form generation, and automatic factual-consistency evaluation. Chapter 3 introduces the technical mechanisms this thesis builds on: the Bradley–Terry preference model, Direct Preference Optimization, the AlignScore factuality metric, the GCA aggregation formula, and the statistical tools used to validate the results. Chapter 4 presents the experimental design, including an explicit account of how the study that was actually carried out diverges from the original proposal, and why. Chapter 5 describes the pipeline, codebase, and compute infrastructure. Chapter 6 reports the full experimental campaign, from the initial reward-model baseline through the formula and judge-mode optimization to the final multi-seed and multi-scale validation. Chapter 7 interprets these results, and Chapter 8 answers the research questions, states the limitations of this work, and outlines directions for future work.

2. Related Work

This chapter reviews four bodies of work that this thesis draws on: preference-based alignment and how preference data becomes a training signal (Section 2.1); the reliability of AI judges when they replace human annotators (Section 2.2); the granularity of feedback in long-form generation, which is the variable this thesis manipulates (Section 2.3); and automatic factual-consistency evaluation, which supplies the judge (Section 2.4). Section 2.5 draws these together into the specific gap this thesis addresses, and Section 2.6 states what separates the present work from the studies closest to it.

2.1. Preference-Based Alignment: RLHF, RLAIF, and Offline Optimization

Preference-based alignment collects pairwise comparisons between candidate model outputs and converts those comparisons into a training signal. The approach originates in preference learning for reinforcement learning, where human annotators compare short trajectory segments and a reward model is fitted to reproduce their comparisons, allowing an agent to be trained on objectives that are easy to judge but hard to specify numerically [3]. The choice to elicit comparisons rather than absolute scores is deliberate: relative judgments between two concrete outputs are more consistent between annotators than absolute ratings on an abstract scale, and the Bradley–Terry model (Chapter 3, Section 3.1) provides a principled way to recover a scalar reward from them.

Stiennon et al. [21] adapted this recipe to summarization, and their pipeline is the direct ancestor of the one used in this thesis. They collect human comparisons between candidate summaries, train a reward model on those comparisons, and then fine-tune a policy against the learned reward with reinforcement learning. Two of their findings matter here. First, the reward model's own held-out agreement with human preferences is reported as a diagnostic in its own right, which establishes the precedent for treating reward-model accuracy as a meaningful measurement rather than merely an intermediate artifact, as this thesis does. Second, they observe that optimizing too hard against a learned reward degrades true quality, an early instance of the reward-overoptimization problem that motivates careful attention to the quality of the preference signal itself. RLHF was subsequently scaled to general instruction-following [15], establishing the reward-model-plus-policy-optimization pattern as the default alignment recipe.

Because human comparison data is expensive, reinforcement learning from AI feedback (RLAIF) substitutes a model for the human annotator. Constitutional AI generates critiques and revisions from a written set of principles and derives preference labels from them, removing human annotation from the harmlessness training loop almost entirely [1]. Lee et al. [12] compare RLAIF and RLHF directly on summarization and dialogue tasks and report broadly comparable downstream performance, which is the empirical result that makes AI-generated preference data a credible substitute rather than a compromise. They also emphasize that the outcome depends substantially on how the judge is prompted and calibrated, a dependency that recurs throughout Section 2.2 and, in a different form, in the evaluator-configuration effects observed here (Chapter 6, Section 6.5).

On the optimization side, RLHF pipelines have typically used on-policy algorithms such as Proximal Policy Optimization [18], which require sampling from the policy during training and maintaining a separate reward model. Direct Preference Optimization (DPO) removes both requirements by reparameterizing the RLHF objective so that the optimal policy can be recovered in closed form directly from preference pairs [17]. DPO was the objective originally proposed for this thesis, chosen because it isolates the effect of the preference data from the dynamics of reinforcement learning. As Chapter 4 describes, DPO was implemented and produced early results before being dropped from the final comparison in favor of evaluating Bradley–Terry reward models directly. That change narrows the comparison further than DPO alone would: it removes both the optimization dynamics that motivated adopting DPO and the decoding variance introduced by generating and scoring a fine-tuned policy's outputs, leaving a more targeted question about whether the preference data itself is more learnable.

2.2. Reliability Risks in AI Supervision and LLM-as-a-Judge

Substituting a language model for a human annotator introduces a distinct class of error. Benchmark studies of LLM-as-a-judge document sensitivity to prompt wording, position bias favoring whichever candidate appears first, verbosity bias favoring longer answers regardless of quality, inconsistent scoring across repeated calls, and self-enhancement effects in which a judge prefers outputs from models similar to itself [24]. Liu et al. [13] document related calibration problems in LLM-based evaluation of generated text, and a recent survey synthesizes the accumulated evidence on these failure modes and the mitigations proposed for them, including rubric enforcement, pairwise rather than absolute evaluation, and position swapping [6].

These issues are more consequential in RLAIF than in evaluation. An unreliable evaluation metric produces a noisy estimate of a model's quality; an unreliable judge used to construct training data produces systematically mislabeled supervision, and the model is then optimized to fit that mislabeling. Casper et al. [2] catalogue failure modes of this kind across the RLHF pipeline, including reward misspecification and

the divergence between training loss and intended behavior, and argue that problems in the preference data propagate into the trained policy in ways that are difficult to detect after the fact.

One response to this literature is to treat the judge as a fallible instrument and wrap it in a reliability protocol: confidence gating, order randomization, evidence-grounded rationale requirements, and prompt-robustness checks. The design adopted here removes the exposure at its source instead, by using a fixed, deterministic factuality model rather than a generative judge (Section 2.4). A deterministic model cannot exhibit prompt sensitivity, position bias, or run-to-run inconsistency, because it is never prompted and returns no natural-language output. This eliminates the category of risk the protocol above is designed to manage, at two costs: it does not make the evaluator correct, and it removes the rationale text that a human audit would examine. Chapter 4, Section 4.3 states what this implies for the claims the thesis can support.

2.3. Feedback Granularity and Credit Assignment in Long-Form Generation

The variable this thesis manipulates is how localized the supervision signal is. A single preference over a multi-sentence output provides one bit of information about a text that may contain a dozen independently checkable claims, and it does not indicate which claim caused the preference. This is a credit assignment problem, and it is present even in the original preference-learning formulation: Christiano et al. [3] have annotators compare short trajectory segments rather than whole episodes precisely to make the resulting supervision denser and easier to attribute.

The closest prior work to this thesis is Wu et al. [22], who study fine-grained human feedback for long-form question answering. Their approach differs from holistic RLHF along two dimensions simultaneously. First, feedback is collected at the density of individual sentences or sub-sentence spans rather than whole responses, so an annotator marks the specific span that is irrelevant, untruthful, or incomplete. Second, they train several distinct reward models, one per error category, and combine their outputs into a dense reward signal during reinforcement learning, so the policy receives a reward at each segment rather than a single scalar at the end of the sequence. They report that this improves both the resulting policy and the reward models themselves relative to holistic feedback, and that the multiple reward models allow the relative weight of different objectives to be adjusted after training.

Three differences separate their setting from this thesis. Their fine-grained labels come from trained human annotators, whereas this thesis asks whether the same localization strategy is still useful when the localized judgments come from an automatic judge, which is cheaper by orders of magnitude but noisier per label. Their fine-grained signal is consumed as a dense per-segment reward during reinforcement learning, whereas GCA aggregates local scores back into a single summary-level pref-

erence pair so that the resulting dataset remains compatible with standard pairwise preference optimization, requiring no change to the training objective. And they vary feedback density and objective decomposition together, whereas this thesis holds the objective fixed at factual consistency alone and varies only granularity, which isolates the granularity question at the cost of addressing a narrower one.

Two more recent lines of work place sentence-level signals at other points in the alignment pipeline, and both should be distinguished carefully from the present study, since neither leaves the question examined here open in general.

Gu et al. [7] introduce Mask-DPO, which incorporates sentence-level factuality signals directly into the DPO objective. Rather than treating a whole response as preferred or dispreferred, Mask-DPO uses sentence-level factuality information to learn only from the factually correct sentences within a preferred response, and to avoid penalizing factually correct content that appears inside a dispreferred one. The motivation is that response-level factuality supervision is noisy, because preferred and dispreferred responses each mix accurate and inaccurate sentences. The intervention point is the policy-optimization objective: the preference pair remains a response-level object, and the granular signal is applied as a mask during training. The present thesis intervenes earlier and leaves the training objective untouched. It asks how automatic factuality judgments at different granularities should be converted into the pairwise label in the first place, and measures the learnability of the resulting labels under a fixed Bradley–Terry architecture.

Qiu et al. [16] modify the reward model itself, proposing a sentence-level reward model that assigns scores at sentence boundaries and aggregates them into a response-level score, while still learning from response-level human preferences. Their aim is to address reward sparsity by producing an intermediate granularity between response-level and token-level reward modeling. The distinction from this thesis is architectural: they change what the reward model computes, and the preference labels they learn from are conventional response-level human judgments. This thesis does not modify the reward-model architecture at all. The Bradley–Terry model, its backbone, and its training procedure are held fixed across both conditions precisely so that differences in outcome can be attributed to the preference labels rather than to the model consuming them.

Reward models are also increasingly evaluated in their own right rather than only as components of a larger pipeline. RewardBench assembles prompt-chosen-rejected trios across several domains and evaluates reward models by how often they rank the preferred response above the dispreferred one [11]. That pairwise-ranking accuracy is the same quantity used as the dependent variable in this thesis (Chapter 4, Section 4.3), which places the measurement on established ground; the difference is that RewardBench varies the reward model against fixed data, whereas this thesis varies the data construction against a fixed reward model.

2.4. Factual Consistency in Summarization and Evaluation

News summarization remains a standard benchmark domain, with CNN/DailyMail widely used for both training and evaluation [8, 14]. Abstractive systems have long produced fluent output that nonetheless contradicts or embellishes the source [19], and this persistent gap between fluency and faithfulness is what makes factual consistency a worthwhile optimization target rather than a solved preprocessing concern.

Automatic factuality evaluation has developed along two main lines. Entailment-based methods treat the source as a premise and the summary as a hypothesis: FactCC trains a classifier on synthetically corrupted summaries to detect factual conflicts [9]. Question-answering-based methods instead generate questions from the summary and check whether the source yields the same answers, an approach developed in FEQA [4] and refined in QAFactEval, which systematically compares components of the QA-based pipeline [5].

Decomposition and aggregation in SummaC. The work most directly relevant to GCA's design is SummaC [10], which observes that natural language inference models trained on sentence-level data transfer poorly to document-level consistency checking, partly because the inputs are far longer than anything seen in training. Their solution is to decompose the problem: they compute an entailment matrix between each source sentence and each summary sentence, then aggregate that matrix into a single score. Critically for this thesis, they show that *how* the matrix is aggregated changes performance substantially. Their zero-shot variant takes, for each summary sentence, the maximum entailment score across all source sentences, and then averages these maxima across summary sentences. Their learned variant instead trains a convolutional layer over the distribution of entailment scores per summary sentence, retaining information that the maximum discards, and performs better.

That aggregation strategy is a design decision with measurable consequences is therefore established in the evaluation literature before this thesis takes it up. The contribution here is to ask the same question one step upstream, at the point where scores are converted into training labels rather than into an evaluation metric, and Chapter 6, Section 6.3 finds the same lesson holds: the choice between a mean and a penalty-weighted aggregation changes the outcome more than the decision to decompose at all.

AlignScore. The judge used throughout this thesis, AlignScore, was not part of the literature surveyed in the original proposal. Zha et al. [23] recast a broad collection of natural language processing tasks, including natural language inference, question answering, paraphrase detection, and fact verification, into a single "information alignment" format, and train one model on the union. The resulting function takes a context and a claim and returns a scalar in $[0, 1]$ estimating whether the context

supports the claim. Because it is trained across many task formats rather than on entailment data alone, it generalizes across factuality benchmarks better than the single-task methods above.

AlignScore also inherits the decomposition question from SummaC, and resolves it internally: in its split evaluation modes, the context is divided into chunks and the claim into sentences, and the final score aggregates the resulting matrix by taking a maximum over chunks and then a mean over claim sentences, which is structurally the same rule as SummaC's zero-shot variant. This internal behavior turns out to matter a great deal for the present work. Chapter 6, Section 6.5 finds that whether GCA outperforms holistic scoring depends on which AlignScore mode is used, and Chapter 7, Section 7.2 argues that this is because the split modes already perform, inside the judge, much of the decomposition that GCA performs outside it.

Fine-grained evaluation. More recent work evaluates summaries at sub-summary granularity rather than assigning a single score. FineSurE assesses faithfulness at the sentence level and content coverage against a list of extracted key facts, producing a diagnostic profile rather than one number [20]. Like SummaC, and unlike this thesis, it is positioned as an evaluation framework: it tells a practitioner where a summary went wrong, but does not convert that localized judgment into supervision for training.

2.5. Synthesis: The Gap This Thesis Addresses

Table 2.1 summarizes the closest lines of work along the dimension that distinguishes the present study: the point in the pipeline at which sentence-level information is introduced, and what each approach holds fixed while doing so.

Table 2.1 organizes this literature by *where* sentence-level information is injected, which is the dimension along which the present study is distinguished. Sentence-level factuality signals are clearly not unexplored: they have been used to mask the policy-optimization objective [7], to restructure the reward model [16], to supply dense rewards during reinforcement learning [22], and to build evaluation metrics [10, 20]. What distinguishes this thesis is the insertion point. Each of the training-oriented works above takes the preference labels as given, in most cases as response-level human judgments, and modifies what happens downstream of them. The step examined here is the construction of those labels: how the output of an automatic factuality evaluator, queried at different granularities, is converted into a pairwise preference, and how learnable the resulting labels are when everything downstream is held constant. This is attractive because automatic labels are cheap, and uncertain because they are noisier per label than human ones, so whether the additional granularity recovers more signal than noise is not obvious in advance.

Table 2.1.: Where sentence-level information enters the pipeline in related work.

Work	Sentence-level signal enters at	Held fixed
Stiennon et al. [21]	not used	–
Lee et al. [12]	not used	–
Wu et al. [22]	reward signal during RL (human labels)	response-level objective
Gu et al. [7]	the DPO objective, as a sentence mask	preference pair construction
Qiu et al. [16]	the reward-model architecture	preference labels (human, response-level)
Laban et al. [10]	evaluation metric only	–
Song et al. [20]	evaluation metric only	–
Zha et al. [23]	internal to the evaluator	–
This thesis	**construction of the preference label**	reward-model architecture and training procedure

2.6. Positioning of This Thesis

The contribution of this thesis is not sentence-level reward modeling itself, nor sentence-level factuality alignment in general, both of which are active areas with established results. It is a controlled comparison of *preference-construction procedures*: the downstream Bradley–Terry architecture and training procedure are held fixed while the granularity at which automatic factuality evidence is converted into pairwise preference labels is varied. No new factuality scorer is introduced; AlignScore is used off the shelf, and the measurement instrument is the learnability of the resulting labels.

Five adjacent lines of work are distinguished as follows:

- **RLAIF and LLM-as-a-judge research** [1, 6, 12, 24] studies judge reliability at the level of a whole-response verdict, rather than the effect of querying the judge at a finer granularity.
- **Fine-grained human feedback** [22] establishes that localized supervision helps when labels are reliable; the present study asks the cheaper and less certain version, in which each local label is produced automatically.
- **Sentence-level factuality alignment** [7] applies granular factuality information inside the policy-optimization objective while leaving preference construction unchanged; here the objective is unchanged and preference construction varies.
- **Sentence-level reward modeling** [16] changes the reward model's architecture while learning from conventional response-level preferences; here the

architecture is fixed and the preferences vary.

- **Fine-grained evaluation frameworks** [10, 20] decompose summaries in order to diagnose them rather than to supervise with them.

The comparison is conducted on preference data generated by one evaluator on one corpus, so the findings describe behaviour under specific conditions rather than a general property of granular supervision (Chapter 7, Section 7.10).

3. Theoretical Background

3.1. Preference Optimization: The Bradley–Terry Model

The Bradley–Terry model is a standard probabilistic model of pairwise comparison. It assumes that every item i has an underlying, unobserved scalar quality r_i, and that the probability of item i being preferred over item j in a pairwise comparison is

$$P(i \succ j) = \sigma(r_i - r_j), \qquad \sigma(x) = \frac{1}{1 + e^{-x}}. \tag{3.1}$$

Given a dataset of observed pairwise preferences (w, l), where w (the "winner") was judged preferable to l (the "loser"), the model parameters are fit by maximizing the likelihood of the observed comparisons under Equation 3.1. Here the parameters belong to a neural network R_θ that maps an (article, summary) pair to a scalar reward, and maximizing the likelihood is equivalent to minimizing the negative log-likelihood

$$\mathcal{L}(\theta) = -\,\mathbb{E}_{(w,l)\sim\mathcal{D}}\Big[\,\log \sigma\big(R_\theta(w) - R_\theta(l)\big)\Big], \tag{3.2}$$

where $\mathcal{D}$ is the preference dataset, holistic or GCA in this thesis. This is exactly the loss implemented in `src/reward_model/train.py` (`bradley_terry_loss`), and it belongs to the same family of reward-modeling loss used inside RLHF pipelines more broadly [15, 21]. The difference here is that the reward model R_θ is not an intermediate component on the way to fine-tuning a generation policy; it is the final object of comparison. The research question this thesis asks, whether GCA produces a more learnable preference signal than holistic scoring, is operationalized directly: does a Bradley–Terry reward model trained on GCA-derived preferences achieve higher held-out pairwise accuracy than one trained on holistic-derived preferences, under an otherwise identical training recipe?

This corresponds to the inverse reinforcement learning perspective: given a set of preference demonstrations, the task is to recover the latent reward function that best explains them. Comparing two reward models trained on two differently constructed preference datasets is then a controlled test of which construction procedure yields the more consistently recoverable latent signal, without requiring a downstream generation policy to be fine-tuned and re-evaluated (Section 3.2).

The architecture used throughout this thesis (`BradleyTerryRewardModel` in `src/reward_model/train.py`) encodes the concatenation of a truncated source article and a candidate summary with a pretrained transformer encoder, mean-pools the encoder's final hidden states over non-padding tokens, and maps the

pooled representation to a scalar reward through a single linear layer. Reward-model accuracy is defined as the fraction of held-out pairs for which the model correctly ranks the chosen summary above the rejected one, i.e. $R_\theta(w) > R_\theta(l)$ (`_evaluate` in `src/reward_model/train.py`); this is the metric reported throughout Chapter 6.

3.2. Direct Preference Optimization (DPO)

This section is included as background on the study's earlier design rather than as part of its final methodology; DPO is not used in the main experiments (Chapter 4, Section 4.7).

Direct Preference Optimization (DPO) replaces RLHF's reward-model-plus-reinforcement-learning pipeline with a single closed-form supervised objective on a policy π_θ, defined relative to a fixed reference policy π_{ref} [17]:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\,\mathbb{E}_{(x,w,l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(w \mid x)}{\pi_{\text{ref}}(w \mid x)} - \beta\log\frac{\pi_\theta(l \mid x)}{\pi_{\text{ref}}(l \mid x)}\right)\right], \tag{3.3}$$

where x is the source article, w and l are the chosen and rejected summaries, and β bounds divergence from the reference policy. The relevant point for this thesis is structural: DPO consumes exactly the same pairwise preference pairs used to train the reward models in Section 3.1, but converts them directly into a policy update, so the quality of the preference data and the behavior of the optimizer both influence the resulting summaries. Separating those two influences is what motivated measuring the preference data on its own (Section 4.7). Early DPO results obtained during the project are reported for context in Chapter 6, Section 6.1.

3.3. AlignScore: Factual Consistency Scoring

AlignScore is a unified alignment function trained across a mixture of natural language processing tasks recast into a single "information alignment" format [23]. Given a context c, the source article in this thesis, and a claim a, a candidate summary or a single sentence from one, AlignScore produces a scalar score in $[0, 1]$ estimating the degree to which a is supported by c. This thesis uses AlignScore unmodified as the sole factuality judge for both preference-construction regimes (`src/judging/reward_model_judge.py`, class `RewardModelJudge`).

AlignScore exposes several evaluation modes, of which four are used in this thesis (the `alignscore-evaluation-mode` argument of `build_reward_preferences.py`):

- `nli_sp`: the context is first split into sentences before scoring, and the claim is scored against this sentence-split representation via AlignScore's NLI (three-way entailment/neutral/contradiction) output head. This was the default mode inherited from the earliest AlignScore-based experiments in the project.

- `nli`: the same NLI output head, without the sentence-splitting preprocessing step.
- `bin_sp` / `bin`: the same two preprocessing variants, but using AlignScore's binary (aligned / not aligned) output head instead of the three-way NLI head.

Chapter 6, Section 6.5 reports an empirical sweep across all four modes. The choice of mode is associated with substantial changes in the outcome, alongside the GCA aggregation formula (Section 3.4). The `nli` mode produced the largest GCA advantage in that sweep and was selected as the final configuration for the validated $n = 1{,}000$ campaign and the scale-robustness checks (Section 6.6). This should be read as a judge-*configuration* finding specific to this pipeline and this task, not as a general property of sentence-level aggregation independent of the underlying judge; Chapter 7 discusses what can and cannot be concluded from it.

3.4. Granular Credit Assignment (GCA)

Granular Credit Assignment (GCA) is the aggregation layer that converts per-sentence AlignScore scores into a single summary-level score, which is then compared against the corresponding score for the other candidate summary to construct a preference label. A candidate summary is first split into sentences with a regex-based sentence splitter (`src/judging/sentence_level.py`, `split_into_sentences`). Each sentence is then scored independently against the full source article using AlignScore (`RewardModelJudge.score_sentences`), yielding a list of per-sentence scores $s = (s_1, \ldots, s_n)$. These per-sentence scores are aggregated into a single summary-level score via (`src/judging/gca.py`, `aggregate_sentence_scores`):

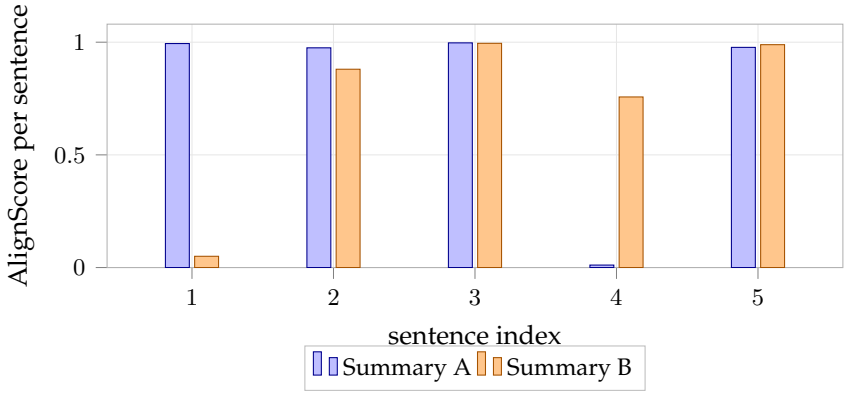
$$\mathrm{gca}(s) = \bar{s} \cdot \left(\frac{\min(s)}{\bar{s}} \right)^{\alpha}, \qquad \bar{s} = \frac{1}{n} \sum_{k=1}^{n} s_k, \tag{3.4}$$

where $\alpha \geq 0$ is a penalty exponent. When $\alpha = 0$, Equation 3.4 reduces to the simple mean, $\mathrm{gca}(s) = \bar{s}$; larger α increasingly penalizes summaries containing at least one low-scoring, "weakest-link" sentence, even when the average sentence score is otherwise high.

Two distinct configurations of Equation 3.4 were used at different points in the project. The original, proposal-era configuration used $\alpha = 0.5$, a moderate weakest-link penalty intended to make GCA sensitive to exactly the failure mode motivating this thesis: a summary that is mostly accurate but contains one fabricated or unsupported sentence. Empirically, this formula achieved only 82.2% agreement with the corresponding holistic decision on the same 1,000 candidate pairs, and when used to construct reward-model training data it caused the GCA reward model to *underperform* the holistic reward model (56.0% vs. 57.2% mean pairwise accuracy; Section 6.2). A subsequent optimization campaign (Section 6.3) tested nine alternative aggregation strategies and found that the simple mean, $\alpha = 0.0$, achieved 97.6%

agreement with holistic decisions, the highest of all strategies tested, and, combined with the AlignScore `nli` mode (Section 3.3), was the configuration that eventually produced the repeated-run GCA advantage (Section 6.6). $\alpha = 0.0$ was accordingly locked as the final configuration.

Figure 3.1 illustrates the difference between the two settings of α on a real candidate pair drawn from the $n = 200$ preference set. Summary A contains four well-supported sentences and one that the evaluator scores at 0.011; summary B is more uniformly mediocre. Under the simple mean the single weak sentence is diluted by the other four and A is preferred, matching the holistic decision. Under the penalty formula the minimum term dominates and the decision reverses. This is the behaviour the penalty was designed to produce, and also the behaviour that made it noisier in practice (Chapter 6, Section 6.3).



Aggregation	Summary A	Summary B	Preferred
Holistic score (full summary)	0.7910	0.7345	A
GCA, $\alpha = 0.0$ (simple mean)	0.7910	0.7345	A
GCA, $\alpha = 0.5$ (penalty)	0.0938	0.1924	B

Figure 3.1.: Worked example on pair `cnn_00233` from the $n = 200$ preference set (evaluator mode `nli_sp`). Summary A has one sentence scored 0.011 among four strong ones. The simple mean preserves the holistic ordering, while the $\alpha = 0.5$ penalty inverts it. Values are computed from the stored per-sentence scores in `data/preferences/gca_reward_preferences_200.jsonl`.

No cross-summary sentence alignment. One design point is worth stating explicitly, since sentence-level methods in this area sometimes align the sentences of one candidate to those of another before comparing them. GCA as defined here performs no such cross-summary alignment. Each candidate summary's sentences are scored independently against the source article, and the two aggregated scores are compared only afterwards. The method therefore contains no alignment step that

could introduce matching errors, and correspondingly offers no opportunity to study alignment quality as a variable.

3.5. Statistical Validation: Bootstrap Confidence Intervals and the Wilcoxon Signed-Rank Test

Two statistical tools are used throughout Chapter 6 to assess whether an observed GCA-versus-holistic accuracy gap is distinguishable from noise.

Bootstrap confidence intervals. Given a set of paired fold-level accuracy differences $\{d_1, \ldots, d_m\}$, where $d_k = \mathrm{acc}_{\mathrm{GCA},k} - \mathrm{acc}_{\mathrm{Holistic},k}$ for cross-validation fold k, pooled across independent training seeds, a bootstrap 95% confidence interval for the mean difference $\bar{d}$ is constructed by resampling $\{d_1, \ldots, d_m\}$ with replacement B times, computing the mean of each resample, and taking the 2.5th and 97.5th percentiles of the resulting distribution of means. This thesis uses $B = 10{,}000$ resamples with a fixed NumPy random seed of 42.

Wilcoxon signed-rank test. As a complementary non-parametric test suited to a small number of paired, non-normally-distributed differences, the two-sided Wilcoxon signed-rank test is reported on the same paired differences. Unlike a paired t-test it does not assume normality, which is appropriate given the small number of observations and the fact that fold accuracies are bounded proportions.

An important qualification. Both procedures assume that the observations they are applied to are independent, and in this thesis they are not. The fold-level differences are pooled across runs, but the five folds within a run partition a single dataset and therefore share most of their training data. Applying either procedure to 30 such observations treats correlated measurements as independent replications, which understates uncertainty and produces intervals that are too narrow and p-values that are too small. Every interval and p-value reported in Chapter 6 is therefore anti-conservative and is presented as an exploratory summary of the spread rather than as a confirmatory hypothesis test. Chapter 7, Section 7.3 discusses what a more conservative treatment would permit and what the results support independently of any threshold.

Pooling across runs rather than reporting a single run is nonetheless deliberate, and responds to an early observation (Section 6.4): a promising single-run result, a $+1.4$ percentage-point advantage from one hyperparameter configuration, did not reproduce when the same configuration was run again. Single runs at this dataset scale are therefore not reliable evidence on their own, whatever statistics are computed from their folds.

4. Methodology

4.1. Research Questions and Hypotheses

- **RQ1.** Does sentence-level factuality feedback aggregated via GCA produce a more learnable pairwise preference signal than holistic summary-level feedback, as measured by reward-model pairwise validation accuracy?
- **RQ2.** How do evaluator configuration and sentence-score aggregation strategy affect the relative learnability of holistic and GCA-derived preferences?
- **RQ3.** How stable are the observed differences across repeated reward-model training runs and dataset sizes?

All three are addressed by the experiments in Chapter 6 and answered in Chapter 8. RQ2 is addressed by an exploratory sweep rather than a controlled ablation, so it supports statements about association between configuration and outcome, not about causal mechanism (Section 4.5).

Three working hypotheses guided the design. H1: sentence-level aggregated preferences are more learnable than holistic preferences, because they localize the factuality signal within a long output. H2: the evaluator's configuration affects whether any such difference is observable. H3: effects observed at one dataset scale need not persist at another. Their status is assessed in Chapter 8.

4.2. Study Design

The comparison is between two procedures for converting factuality scores into pairwise preference labels. For every source article, two candidate summaries are generated once and reused by both conditions:

Holistic. The complete candidate summary is scored against the source article, producing one score per candidate. The two scores are compared to yield a pairwise preference.

GCA. The candidate summary is segmented into sentences; each sentence is scored separately against the source article; the sentence scores are aggregated into one summary-level score. The two aggregated scores are compared to yield a pairwise preference.

The following are held identical across conditions: the source articles; the two candidate summaries and the decoding settings that produced them; the evaluator family and checkpoint; the reward-model architecture; and the reward-model training and evaluation procedure. The manipulated variable is the granularity at which the evaluator is queried, together with the aggregation step that GCA requires and holistic scoring does not. Table 4.1 lists the final configuration.

Table 4.1.: Final experimental configuration.

Factor	Value
Task	Single-document abstractive summarization (news)
Dataset	CNN/DailyMail; runs at $n = 1{,}000$, $5{,}000$, and $10{,}000$ (subset seed 200)
Candidate generation	Mistral-7B-Instruct-v0.3, two-temperature sampling ($T = 0.7$ / $T = 1.0$)
Evaluator	`yzha/AlignScore`, mode `nli`
Manipulated variable	Feedback granularity: holistic vs. sentence-level + GCA
GCA aggregation	Simple mean ($\alpha = 0.0$)
Margin	0 (all pairs used)
Reward model	Bradley–Terry, `FacebookAI/roberta-base` backbone, mean-pool + linear scalar head
RM training	epochs = 5, lr = 2×10^{-5}, batch size = 8, max length = 512, 5-fold cross-validation
Dependent variable	Reward-model pairwise validation accuracy

4.3. Scope of the Evaluation

The dependent variable throughout this thesis is the pairwise validation accuracy of a Bradley–Terry reward model trained on a given preference set. Within this thesis, *learnability* denotes exactly this quantity: the extent to which the preference relation encoded in a dataset can be recovered by the chosen reward-model architecture under a fixed training procedure.

Because this term is easily over-read, it is worth stating precisely what a higher accuracy does and does not establish. A higher value indicates that the generated preference relation is more consistently recoverable by the selected reward-model architecture. It does not by itself establish any of the following:

- that the preference labels agree better with human factuality judgments;
- that the labels are objectively more accurate as factuality annotations;
- that a summarization model trained on those preferences produces more factually consistent summaries;

- that GCA is generally preferable to holistic scoring.

The first two are inaccessible here because no independent human annotation was collected, so the labels produced by AlignScore function as the reference standard rather than as verified ground truth. The third is inaccessible because the final study does not include a downstream policy-optimization stage (Section 4.7). The fourth is addressed empirically and answered in the negative: Chapter 6 reports configurations in which GCA does not lead. Claims made in later chapters are confined to learnability in the sense defined here.

4.4. Reward-Model Evaluation Protocol

Reward-model accuracy is evaluated by 5-fold cross-validation rather than a single fixed held-out split. At the dataset sizes used here, a fixed split would leave either too little data for training or too little for a stable accuracy estimate, whereas cross-validation uses the full dataset for both and yields five accuracy readings per run.

Cross-validation alone does not account for variance introduced by the reward model's initialization and by which pairs fall into which fold. This became apparent during the study: a first run of the final configuration produced a GCA advantage of $+6.0$ percentage points, while a rerun of the identical configuration *at the same seed value* produced $+1.3$ percentage points (Chapter 6, Section 6.5). That two same-seed runs diverge follows from an implementation detail documented in Chapter 5, Section 5.5: the `-seed` argument controls the cross-validation fold assignment, while PyTorch's generator is left unseeded, so weight initialization, batch ordering and dropout all vary between executions. A seed value therefore does not pin down a run completely, and repeated runs at one seed value are not redundant.

The $n = 1{,}000$ result is accordingly pooled over *six training runs* spanning *five distinct seed values* (42, 42, 7, 100, 314, 2026; seed 42 appears twice, as the original run and its rerun), giving 30 fold-level accuracy readings per condition.

Two properties of this pooling constrain the statistical claims that can be made from it, and are revisited in Chapter 7, Section 7.3. Six runs across five seed values is a modest basis for a variance estimate. More consequentially, the 30 fold-level differences are not 30 independent observations: the five folds within a run partition a single dataset, so their training sets overlap heavily and their errors are correlated. Procedures assuming independent observations therefore overstate the effective sample size. Fold-level uncertainty estimates are reported in Chapter 6 as exploratory and anti-conservative, not as confirmatory tests.

The $n = 5{,}000$ and $n = 10{,}000$ runs were each executed once rather than repeated, for reasons of computational budget, and carry correspondingly less weight than the pooled $n = 1{,}000$ result.

4.5. Exploratory and Confirmatory Experiments

The experiments in Chapter 6 do not all have the same evidential status, and the distinction is material to how the results should be read.

The aggregation-formula comparison (Section 6.3), the evaluator-mode sweep (Section 6.5), and the hyperparameter search (Section 6.4) were all carried out on the $n = 1{,}000$ setting, and the final configuration was selected after observing their outcomes. These experiments are exploratory. The configuration they produced was not specified in advance, and reporting it as though it had been pre-registered would overstate the evidence: with enough configurations examined, some separation between conditions is expected by chance alone.

The repeated runs at $n = 1{,}000$ (Section 6.6) test whether the selected configuration produces a stable effect under repetition, but they reuse the dataset on which that configuration was chosen and therefore cannot establish that it generalizes beyond that setting.

The $n = 5{,}000$ and $n = 10{,}000$ experiments provide a weaker check than would be ideal, for a reason that must be stated explicitly. All three subsets are drawn from the same CNN/DailyMail test split by the same deterministic procedure at the same seed, differing only in the number of samples requested, and they are consequently nested rather than disjoint (Section 4.6). The larger-scale runs therefore re-use almost all of the data on which the configuration was selected, and are best understood as measurements of how the effect behaves as the dataset grows, not as out-of-sample replication on independent data. No experiment in this thesis evaluates the selected configuration on data disjoint from the set used to choose it.

4.6. Relationship Between the Dataset Sizes

The three dataset sizes used in the final experiments are not independent samples, and this constrains how the larger-scale runs can be interpreted.

All three subsets are produced by the same deterministic procedure (`src/data/subset.py`, `select_subset`) from the same CNN/DailyMail test split, under the same length filters and the same subset seed (200). Only the number of requested samples differs. Because the selection draws from one eligible pool of 10,896 articles with a fixed random seed, the resulting subsets overlap heavily. Reproducing the selection with the recorded parameters gives the intersections in Table 4.2.[1]

The $n = 5{,}000$ subset is contained entirely within the $n = 10{,}000$ subset, and the $n = 1{,}000$ subset is almost entirely contained within both. The $n = 10{,}000$ set

[1] Computed by re-running `select_subset` with the parameters recorded in `configs/subset_1000.yaml` and `slurm/generate_candidates_scale.sh` (seed = 200, test split, article limit 8,000 characters, summary limit 2,000 characters) against the local copy of the test split in `data/cnn_dailymail_test_full`, and comparing the resulting SHA-256-derived sample IDs. The candidate and preference files for these runs are not retained in the repository, so the comparison reproduces the selection rather than reading the stored artifacts; the selection is deterministic given these parameters.

Table 4.2.: Overlap between the subsets used in the final experiments. All three use subset seed 200 and differ only in sample count.

Pair	Shared sample IDs	Share of the smaller set
$n = 1{,}000$ and $n = 5{,}000$	980	98.0%
$n = 1{,}000$ and $n = 10{,}000$	999	99.9%
$n = 5{,}000$ and $n = 10{,}000$	5,000	100%

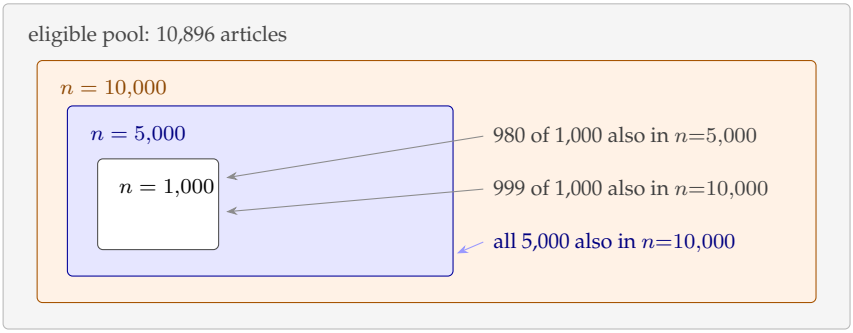


Figure 4.1.: The three subsets are nested rather than independent. Box sizes are schematic; the counts are the measured intersections reported in Table 4.2.

additionally covers 91.8% of the whole eligible pool, so at that size little of the test split remains unused in any case.

Two consequences follow, and both are carried through the rest of the thesis. The larger-scale experiments do not constitute out-of-sample replication: they re-use almost all of the data on which the configuration was selected, and therefore measure how the effect behaves as further data is added to the same pool rather than whether it transfers to new data. And because the samples are nested, the three results are not statistically independent of one another, so the differences between them should not be read as three separate estimates of the same quantity. Drawing the larger subsets with different seeds would have made them independent, at the cost of the nesting property the generation script was written to provide; this is recorded as a limitation in Chapter 8.

4.7. Evolution of the Study Design

The study reported here is narrower in scope than the one outlined in the original research plan, and the reasons are methodological.

The initial design used the generated preferences directly for DPO-based policy optimization, evaluating factual consistency on the fine-tuned model's own generations. Early downstream experiments made it difficult to attribute observed differences to any single stage: a change in generated-summary factuality could originate in

the preference-construction procedure, in the quality of the reward signal, or in the policy-optimization step, and the design provided no way to separate these. Increasing the number of stages also increased the number of variance sources standing between the manipulated variable and the measured outcome.

The final study therefore isolates an earlier and more controlled diagnostic question: whether the preference data produced by each construction procedure is recoverable by a fixed reward-model architecture. Removing the policy-optimization stage removes both reinforcement-learning dynamics and decoding variance from the measurement path, leaving a comparison in which granularity is the principal difference between conditions.

The claims of this thesis are correspondingly narrower. The work evaluates the learnability of the preference signal rather than demonstrating improvements in generated-summary factuality (Section 4.3). Two further consequences follow from the same change. Margin-based filtering of near-tie pairs was removed, since AlignScore produces continuous scores for which exact ties are effectively impossible, and the threshold discarded roughly one fifth of the data without a demonstrated benefit. The reliability protocol designed for a generative LLM judge, comprising confidence gating, order randomization, and rationale requirements, no longer applies once the evaluator is a deterministic model that is never prompted and returns no rationale; a planned human audit of judge rationales was not carried out and is recorded as a limitation in Chapter 8.

4.8. Validity Considerations

Ethical and practical considerations. All experiments use CNN/DailyMail, a publicly available corpus of news articles and human-written summaries; no personally sensitive data is collected. Because both the evaluator's judgments and the generated summaries can be incorrect, results are reported as measurements of preference-data learnability rather than as statements about factual correctness, and model outputs are not presented as verified fact. Compute use was bounded by working with fixed-size subsets.

Candidate generation and decoding temperature. The two candidates in each pair are produced by the same model at different decoding temperatures ($T = 0.7$ and $T = 1.0$). This introduces a potential confound: preference structure may correlate with systematic stylistic differences induced by temperature rather than with factuality alone. The preference data stored in the repository allow the asymmetry to be quantified directly, and Table 4.3 reports it.[2]

[2]Computed from `data/preferences/holistic_reward_preferences_200.jsonl` and `data/preferences/gca_reward_preferences_200.jsonl`. The margin-0 rows recompute the decision from the stored scores at the threshold used in the final configuration; the margin-0.05 rows reproduce the stored decisions, whose aggregate counts also appear in `reports/reward_model_judging_summary.csv`. Preference files for the final $n = 1{,}000$ configuration are not

Table 4.3.: Proportion of pairs in which the lower-temperature candidate ($T = 0.7$) is preferred, on the $n = 200$ preference set (evaluator mode `nli_sp`, $\alpha = 0.5$).

Condition	Threshold	T=0.7 preferred	T=1.0 preferred	Share
Holistic	margin $= 0$	137	63	68.5%
GCA	margin $= 0$	126	74	63.0%
Holistic	margin $= 0.05$	109	45	70.8%
GCA	margin $= 0.05$	97	60	61.8%

The asymmetry is substantial under both conditions and consistently larger under holistic scoring, and it is not an artifact of the margin threshold. Two implications follow. Features correlated with decoding temperature, such as summary length or lexical conservativeness, may carry a non-trivial share of the label information, so a reward model could recover part of the preference relation without representing factuality as such. And because the asymmetry differs between conditions by roughly five percentage points, the two preference sets are not equivalent in this respect either, which means the comparison between them is not perfectly clean with respect to temperature.

These figures come from the $n = 200$ set scored under the earlier evaluator configuration and need not hold exactly at the final one. They are reported as evidence that the confound is real and measurable, not as a quantification of its effect on the reported results. Controlling candidate generation more explicitly, for instance by sampling both candidates at one temperature, or testing whether simple candidate-level features such as length predict the labels, is left to future work (Chapter 8).

Threats to validity. Table 4.4 summarizes the threats considered and how each is handled.

retained in the repository, so these proportions could not be recomputed under the final evaluator mode.

Table 4.4.: Threats to validity and how each is addressed.

Threat	Status
Single evaluator (AlignScore)	Not mitigated; all results are conditional on this evaluator's judgments, which are not independent ground truth.
Evaluator and aggregation sensitivity	Measured directly via the formula comparison and evaluator-mode sweep (Chapter 6).
Fold- and run-level noise	Addressed by 5-fold cross-validation pooled over six runs for the $n = 1{,}000$ result (Section 4.4).
Non-independence of pooled folds	Acknowledged, not mitigated; fold-level uncertainty estimates are anti-conservative (Section 4.4).
Configuration selected on observed results	Acknowledged, not remedied; the larger-scale subsets are nested supersets of the selection data (Section 4.6), so no run evaluates the configuration on independent data.
Scale sensitivity	Examined explicitly rather than assumed away (Section 6.7).
Decoding-temperature confound	Quantified where data permit; not controlled for in the design (see above).
Source-document truncation	Documented in Chapter 5, Section 5.5; discussed as a limitation.
Weak absolute accuracy	Both conditions remain near chance (Chapter 7, Section 7.4).

5. Implementation

5.1. Pipeline Overview

Figure 5.1 shows the final pipeline. It differs from the pipeline described in the original proposal in exactly the ways described in Chapter 4: there is no DPO fine-tuning stage, no MoDPO branch, and no cross-summary sentence-alignment step. The pipeline has four stages. First, a fixed number of CNN/DailyMail articles are selected deterministically (Section 5.2). Second, two candidate summaries are generated per article at different sampling temperatures (Section 5.3). Third, the same candidate pool is scored twice, once holistically and once at the sentence level with GCA aggregation, to produce two separate preference datasets (Section 5.4). Fourth, a Bradley–Terry reward model is trained independently on each preference dataset and the two are compared (Section 5.5).

5.2. Data

All experiments use CNN/DailyMail [8], version `3.0.0` as distributed through the Hugging Face `datasets` library. Subset selection (`src/data/subset.py`, function `select_subset`) filters the chosen split to articles under 8,000 characters and reference summaries under 2,000 characters, then draws a fixed-size sample from the eligible pool using Python's `random.Random` seeded deterministically. Each sample is assigned an ID of the form `cnn_{index}_{hash}`, where `hash` is the first 12 hexadecimal characters of the SHA-256 digest of the article text (`make_sample_id`). This ties every sample's identifier to its content rather than to its position in the dataset, so the same article always receives the same ID regardless of which subset size or seed selected it.

Table 5.1 lists every subset used across the project. The $n = 1{,}000$, $n = 5{,}000$, and $n = 10{,}000$ subsets all use subset seed 200 and the same filtering parameters, differing only in the number of samples requested. They are therefore *nested rather than disjoint*: the smaller sets are almost entirely contained in the larger ones. Chapter 4, Section 4.6 quantifies the overlap and states its methodological consequence. Earlier subsets at seeds 42 and 100 were drawn with different seeds and are not part of the final comparison.

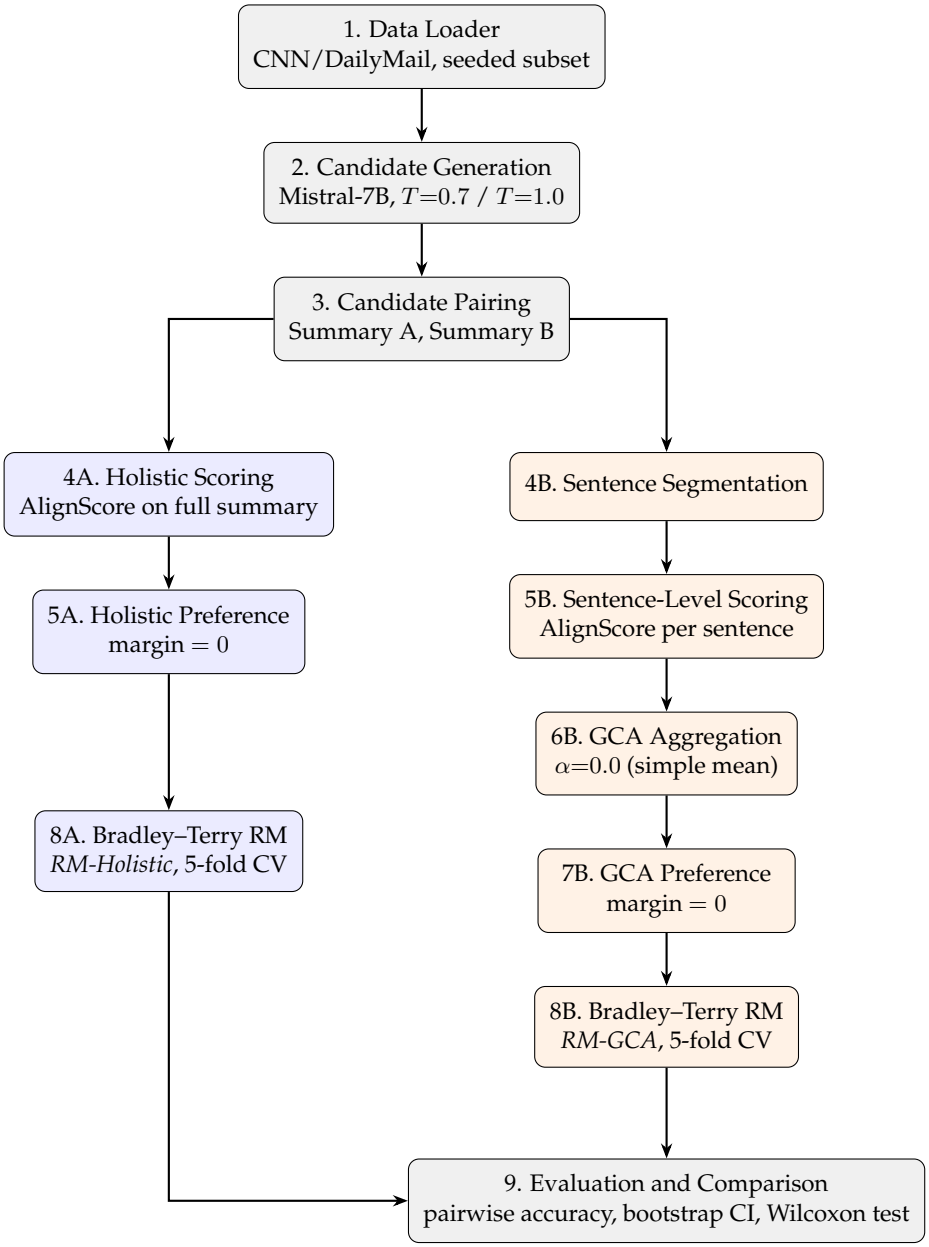


Figure 5.1.: The final pipeline. Shared setup (grey) produces one candidate pool from which two independent preference-construction branches run: holistic (blue) and GCA (orange). Each branch trains its own Bradley–Terry reward model under an identical recipe, and the two are compared in the final evaluation step. Compare to the pipeline described in the original proposal (Chapter 4), which additionally included a DPO fine-tuning stage and a cross-summary sentence-alignment step, neither of which appears here.

Table 5.1.: CNN/DailyMail subsets used across the project.

n	Subset seed	Purpose	Notes
200	42	Early DPO-based prototype	Superseded; see Chapter 6, Section 6.1
495	100	First Bradley–Terry RM training set	Drawn with a different seed from the $n = 200$ set
1,000	200	Final repeated-run campaign	6 RM training runs; seeds 42, 42, 7, 100, 314, 2026
5,000	200	Scale-robustness check	Single training run
10,000	200	Scale-robustness check	Single training run

5.3. Candidate Generation

Candidate summaries are generated with Mistral-7B-Instruct-v0.3, loaded in `bfloat16` without quantization (`src/generation/candidates.py`). On the A100-SXM4-40GB GPUs used for this project, the model occupies roughly 14 GB in `bfloat16`, comfortably within budget, so 4-bit quantization, though supported in the code via `BitsAndBytesConfig`, is disabled in the configuration actually used (`load_in_4bit: false` in `configs/generation.yaml`). The tokenizer uses left-padding, required for batched generation with a decoder-only model, and articles are truncated to 1,024 input tokens.

Each article is summarized twice, using the fixed prompt template `"Summarize the following news article in a concise paragraph. ..."`, with two different sampling configurations: a low-temperature setting ($T = 0.7$, top-$p = 0.9$) and a high-temperature setting ($T = 1.0$, top-$p = 0.95$), generating up to 256 new tokens each. These two outputs form the candidate pair ("summary A" and "summary B") that both the holistic and GCA preference construction stages score. Generation is resumable: the script checks which sample IDs already appear in the output file and only generates candidates for the remainder, which matters in practice on a shared cluster where a job may need to be resubmitted after a queue timeout.

5.4. Preference Construction

Both preference-construction regimes are implemented in `src/judging/build_reward_preferences.py` and share a single judge object, `RewardModelJudge` (`src/judging/reward_model_judge.py`), configured to use the AlignScore backend.

In the holistic regime (`judge_holistic_pair`), each full candidate summary

is scored once against the article with `judge.score(article, summary)`, and the two resulting scores are compared with a margin guard (`make_decision`): summary A is preferred if its score exceeds summary B's by more than the margin, summary B is preferred in the opposite case, and the pair is marked `no_preference` otherwise. With the final margin of 0 (Chapter 4, Section 4.7), every pair with a nonzero score difference produces a decision.

In the GCA regime (`judge_gca_pair`), each candidate summary is first split into sentences with a regex-based splitter, then every sentence is scored independently against the article with `judge.score_sentences(article, sentences)`. The resulting list of per-sentence scores is passed to `aggregate_sentence_scores(scores, alpha)` (Chapter 3, Equation 3.4) to produce a single GCA score per summary, and the same margin-guarded comparison determines the preference. Every output record retains the full sentence-level detail, including each sentence's individual AlignScore score and a count of "low-faithfulness" sentences (score below 0.5), which supports the qualitative disagreement analysis in Chapter 6.

The locked final configuration is invoked as:

```
python src/judging/build_reward_preferences.py
  -mode both -alpha 0.0 -margin 0
  -alignscore-evaluation-mode nli
  -alignscore-backbone FacebookAI/roberta-base
```

5.5. Reward-Model Training

Reward-model training is implemented in `src/reward_model/train.py` (`BradleyTerryRewardModel`, `bradley_terry_loss`) and orchestrated by `src/reward_model/run_training.py`, which trains the holistic and GCA conditions from a single invocation. All results reported in Chapter 6 use k-fold cross-validation (`_kfold_cv` in `run_training.py`): the preference pairs are shuffled using a seeded `random.Random` instance, split into $k = 5$ contiguous folds, and for each fold a freshly initialized reward model is trained on the other four folds and evaluated on the held-out one. One implementation detail is worth stating precisely, since it affects how "training seed" should be interpreted. The `-seed` argument is passed to Python's `random` module, which governs the fold-assignment shuffle. PyTorch's own generator is not seeded anywhere in the cross-validation path (`_kfold_cv`), so every source of stochasticity drawn from it is uncontrolled: the randomly initialized reward head, the shuffling of training batches by the `DataLoader`, and the dropout masks applied during training. Changing `-seed` therefore changes the fold assignment, while these other sources vary between process executions regardless of the seed value. Together they explain why two executions launched with the same seed value produced different results (Chapter 6, Section 6.6), and they are discussed as a source of run-to-run variation in Chapter 7, Section 7.8.

The final training configuration, used for every reported reward-model result, is: `FacebookAI/roberta-base` backbone, 5 epochs, batch size 8, learning

rate 2×10^{-5} with a linear warmup over 10% of total steps, maximum sequence length 512 tokens (article truncated to 2,000 characters before tokenization, concatenated with the summary), and 5-fold cross-validation. This is confirmed by `slurm/train_rm_1000.sh`, the Slurm script used for the headline $n = 1{,}000$ result. One inconsistency in the codebase is worth flagging explicitly, since it could otherwise look like an error in the reported results: the in-code default for `–backbone` in both `train.py` and `run_training.py` is `microsoft/deberta-v3-base`, a leftover from an earlier version of the training script. Every Slurm script used to produce a result reported in this thesis overrides this default and passes `–backbone FacebookAI/roberta-base` explicitly; the `deberta-v3-base` default was never actually used to produce a reported number. The one exception is the very first Bradley–Terry result on the $n = 495$ set (Chapter 6, Section 6.2), which predates the standardization on `roberta-base` and is reported as such.

Source-document truncation. The reward model does not see the full source article. Two limits apply in sequence. The article string is first truncated to its leading 2,000 characters (`–max-article-chars`, applied in `PreferencePairDataset`), and the truncated article is then concatenated with the candidate summary and tokenized under a 512-token cap (`–max-length`), so the article prefix is shortened further to make room for the summary. Subset selection admits articles up to 8,000 characters (Section 5.2), so for the longer articles in the pool the reward model observes well under half of the source text.

This matters for interpreting the reward-model results. The evaluator that produced the preference labels scores summaries against the article under its own chunking scheme, whereas the reward model must reconstruct those labels from a prefix only. Where the evidence supporting or contradicting a claim lies beyond the retained prefix, that evidence is simply unavailable to the reward model, and the corresponding preference is not recoverable from its input in principle rather than merely in practice. How large this effect is was not measured here, and the near-chance accuracies reported in Chapter 6 should not be attributed to truncation without evidence; it is one plausible contributing factor among several discussed in Chapter 7, Section 7.4. Varying `–max-article-chars` while holding everything else fixed would isolate it, and is recorded as future work.

5.6. Compute Infrastructure

All experiments run on MOGON NHR (`mogon-nhr-01.zdv.uni-mainz.de`), the University of Mainz's high-performance computing cluster, using Slurm partition `a100dl` under account `nhr-haloed`, on NVIDIA A100-SXM4-40GB GPUs. All Slurm scripts producing results reported in Chapter 6 exclude one cluster node found to have a faulty CUDA runtime, via `#SBATCH –exclude`; this affects scheduling only and not the computation performed.

5.7. Reproducibility

The pipeline is config-driven throughout: dataset, generation, and judging parameters live in `configs/*.yaml` and can be overridden per run from the command line, and every script accepts explicit flags for the parameters that matter for a given experiment (margin, α, AlignScore evaluation mode, reward-model hyperparameters, and so on) rather than hardcoding them.

Seeding is partial, and the limits should be stated exactly. Subset selection and cross-validation fold assignment are explicitly seeded through Python's `random` module. PyTorch's generator is not seeded in the cross-validation path, so weight initialization, batch ordering and dropout are not fixed, and exact run-level reproduction from the command-line seed alone is not guaranteed: two executions with identical arguments can differ. Every pipeline script writes a JSON run-metadata file (`src/utils/logging.py`, `save_run_metadata`) recording its configuration, output artifact paths, and a timestamped run ID, so that any reported number can in principle be traced back to the exact invocation that produced it.

The configuration required to reproduce the $n = 1{,}000$ result is given in full in Table 4.1 (Chapter 4), together with the six training seeds listed in Chapter 6, Table 6.5, and is implemented by `slurm/train_rm_1000.sh` and `slurm/mode_nli_seed_confirm.sh`. The codebase, including all scripts referenced in this chapter, is available at the project's GitHub repository: `https://github.com/hasnat23/master-thesis-rlaif-gca`.

6. Results

This chapter reports the experiments in the order they were carried out. Two intermediate findings did not survive repetition: a hyperparameter configuration that initially appeared promising (Section 6.4) and a two-run estimate that proved inconclusive (Section 6.5). Both are reported, since they motivated the repeated-run protocol described in Chapter 4, Section 4.4, and omitting them would misrepresent how much of the final estimate rests on distinguishing an effect from run-to-run variance. Sections 6.3 to 6.5 are exploratory in the sense of Chapter 4, Section 4.5; Section 6.7 examines how the selected configuration behaves as the dataset grows.

6.1. Early Prototype Results (Historical Context)

An earlier version of the pipeline produced results under a DPO-based design before the study narrowed to reward-model comparison (Chapter 4, Section 4.7). They are reported here as context for that change, not as findings of the present study.

A FLAN-T5 baseline on a 20-sample pilot gave ROUGE-1 0.3278, ROUGE-2 0.1201, ROUGE-L 0.2341, and BERTScore F1 0.8612. On a 200-sample subset (subset seed 42), the original AlignScore-based preference construction, using the proposal-era margin of 0.05 and $\alpha = 0.5$, produced 154 usable holistic pairs out of 200 (77.0%, mean scores 0.7324 for the low-temperature candidate and 0.6530 for the high-temperature one) and 157 usable GCA pairs (78.5%, mean GCA scores 0.4066 and 0.3337). The two methods agreed on 121 of 200 decisions (60.5%).

Mistral-7B-Instruct-v0.3 was then fine-tuned with DPO on both preference sets (Chapter 3, Section 3.2): training loss reached 0.6902 for the holistic condition and 0.6913 for the GCA condition, each in roughly 134.5 seconds on an A100. On 200 held-out articles, baseline AlignScore was 0.7965, DPO-Holistic reached 0.8017, and DPO-GCA reached 0.7996. Only DPO-GCA's ROUGE-L differed from baseline at the conventional threshold (Wilcoxon $p = 0.015$); the AlignScore differences between conditions were small and not distinguishable from noise. These downstream results illustrate the attribution problem described in Chapter 4, Section 4.7: differences of this magnitude cannot be traced to the preference-construction procedure rather than to the optimization stage, which is why the final study measures the preference data directly.

6.2. Bradley–Terry Reward-Model Baseline

The first Bradley–Terry reward-model result used the $n = 495$ set (subset seed 100) with a `microsoft/deberta-v3-base` backbone, predating the standardization on `FacebookAI/roberta-base` (Chapter 5, Section 5.5). Under 5-fold cross-validation, the holistic reward model reached 58.1% mean pairwise accuracy and the GCA reward model reached 54.6%, both above the 50% chance level.

The first result on the $n = 1{,}000$ set, with the `roberta-base` backbone and the original configuration ($\alpha = 0.5$, margin 0.05), is summarized in Table 6.1. This is the baseline the rest of this chapter's optimization campaign works from: holistic outperforms GCA by 1.2 percentage points.

Table 6.1.: Bradley–Terry reward-model baseline at $n = 1{,}000$ ($\alpha = 0.5$, margin 0.05, `roberta-base`).

Condition	Mean pairwise accuracy	Gap (GCA − Holistic)
Holistic RM	57.20%	-1.20 pp
GCA RM	56.00%	

6.3. GCA Formula Optimization

To understand why GCA underperformed, disagreement between the holistic and GCA decisions at $\alpha = 0.5$ was analyzed directly on the $n = 1{,}000$ pairs (`analysis/gca_vs_holistic_analysis.py`). The two methods disagreed on 17.8% of pairs. GCA scores showed higher discrimination between candidates than holistic scores (mean absolute score difference 0.2392 versus 0.1527), but only 82.2% of GCA decisions agreed with the corresponding holistic decision, suggesting that the extra discrimination was at least partly noise rather than signal.

Nine aggregation strategies were then compared by how well each agreed with the holistic decision on the same 1,000 pairs (`analysis/test_gca_formulas.py`, `analysis/test_advanced_aggregation.py`). Table 6.2 reports the results. The simple mean ($\alpha = 0.0$) had the highest agreement of any strategy tested, 15.4 percentage points above the original penalty formula.

On the strength of this result, the default α was changed from 0.5 to 0.0 throughout the codebase (`src/judging/gca.py`, `src/judging/build_reward_preferences.py`, and every Slurm script).

Two qualifications apply to this selection. First, the criterion used in Table 6.2 is agreement with the holistic decision, which is a measure of internal consistency between two procedures driven by the same evaluator, not a measure of external factual correctness. A strategy could agree closely with holistic scoring while reproducing the same errors, and nothing in this comparison would distinguish those

Table 6.2.: Agreement with the holistic decision across nine GCA aggregation strategies, $n = 1{,}000$ pairs.

Strategy	Agreement with holistic	vs. $\alpha = 0.5$ baseline
Simple mean ($\alpha = 0.0$)	97.6%	+15.4 pp
Confidence-weighted	96.0%	+14.0 pp
Ensemble	93.3%	+11.1 pp
Length-weighted	91.5%	+9.3 pp
Weighted by position	88.2%	+6.0 pp
Robust mean (10% trim)	85.4%	+3.2 pp
Harmonic mean	82.9%	+0.7 pp
Original penalty ($\alpha = 0.5$)	82.2%	baseline
Minimum	78.7%	−3.5 pp

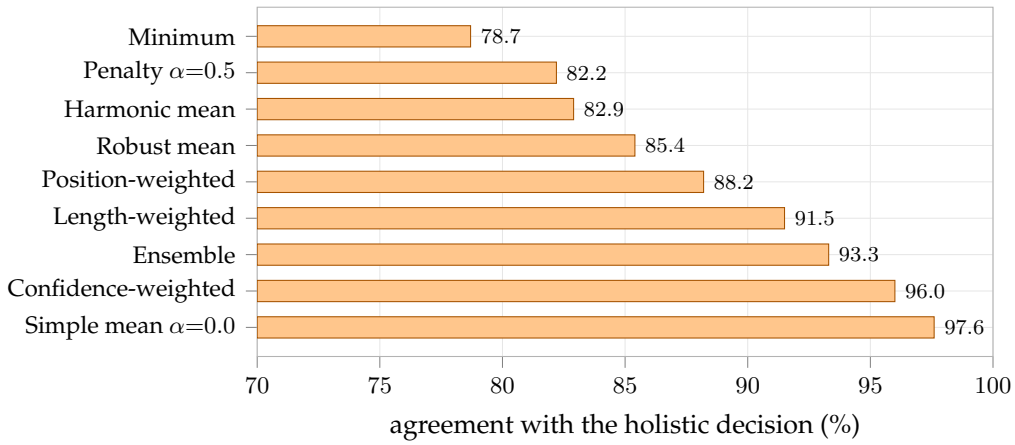


Figure 6.1.: Agreement with the holistic decision across the nine aggregation strategies of Table 6.2 ($n = 1{,}000$ pairs). Agreement measures consistency between two procedures driven by the same evaluator, not external factual correctness.

cases. The criterion is used here as a diagnostic for whether the penalty term was amplifying disagreement, not as evidence that the simple mean produces better labels. Second, this comparison and the evaluator-mode sweep that follows are exploratory in the sense defined in Chapter 4, Section 4.5: the final configuration was chosen after observing $n = 1{,}000$ outcomes, so results on that setting cannot also serve as independent confirmation of the choice. The $n = 5{,}000$ and $n = 10{,}000$ experiments do not fully remedy this, because their subsets are nested supersets of the $n = 1{,}000$ set rather than independent samples (Chapter 4, Section 4.6).

6.4. Does the Formula Change Alone Flip the Result?

The formula change was then validated the same way the baseline was measured: by training reward models on the resulting preferences and checking pairwise accuracy, holding the judge mode fixed at the original `nli_sp` setting. Table 6.3 shows that $\alpha = 0.0$ did not, on its own, close the gap. Holistic accuracy improved more than GCA accuracy did, and the gap between them widened slightly rather than closing.

Table 6.3.: Effect of the formula change alone ($n = 1{,}000$, judge mode `nli_sp` unchanged).

Metric	$\alpha = 0.5$ (baseline)	$\alpha = 0.0$	Change
Holistic mean accuracy	0.572	0.586	$+0.014$
GCA mean accuracy	0.560	0.561	$+0.001$
Gap (Holistic − GCA)	0.012	0.025	$+0.013$ (wider)

A nine-configuration hyperparameter search was run next, sweeping learning rate over $\{1 \times 10^{-5}, 2 \times 10^{-5}, 5 \times 10^{-5}\}$ and epoch count over $\{3, 5, 7\}$, still at $\alpha = 0.0$. One configuration, learning rate 1×10^{-5} with 7 epochs, produced a GCA advantage of $+0.014$ (GCA 0.586 versus holistic 0.572). A same-seed confirmation rerun of that exact configuration, however, produced the opposite result: holistic 0.577 versus GCA 0.560, a gap of -0.017. The apparent advantage did not reproduce. This result is reported in full because of what it implies: at this dataset size, a single training run, even with 5-fold cross-validation, can produce a result that looks like a real effect but is actually within the range of ordinary fold-to-fold and initialization noise. This directly motivated moving away from hyperparameter tuning toward the judge-mode sweep below, and toward the multi-seed validation protocol used for the headline result.

6.5. AlignScore Judge-Mode Sweep

With $\alpha = 0.0$ fixed, four AlignScore evaluation modes (Chapter 3, Section 3.3) were compared under the same $n = 1{,}000$ training setup. Table 6.4 shows that the choice

of mode, not only the aggregation formula, is associated with whether GCA leads at all. The `nli` mode produced the largest GCA advantage of any configuration tested so far; `nli_sp`, the mode used for every result up to this point, is actually one of the two modes that favor holistic.

Table 6.4.: AlignScore judge-mode sweep ($n = 1{,}000$, $\alpha = 0.0$).

Mode	Holistic mean acc.	GCA mean acc.	Gap (GCA − Holistic)
`nli`	0.523	0.583	+0.060
`bin`	0.510	0.564	+0.054
`bin_sp`	0.554	0.562	+0.008
`nli_sp`	0.578	0.557	−0.021

Figure 6.2 plots the same values grouped by whether the evaluator performs its own internal splitting.

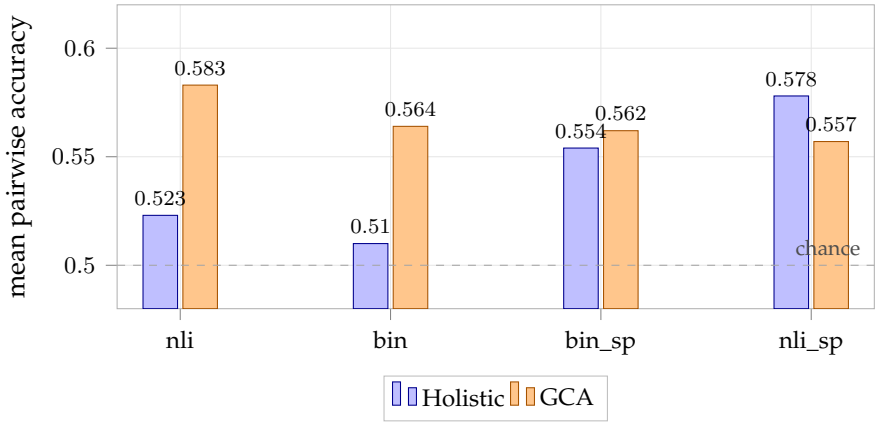


Figure 6.2.: Evaluator-mode sweep ($n = 1{,}000$, $\alpha = 0.0$). The two left-hand modes do not split the input internally; the two right-hand modes do. Each bar is a single run, so differences of this size fall within the run-to-run variation documented in Section 6.6, and the comparison is exploratory.

The ordering in Table 6.4 follows a systematic pattern rather than varying arbitrarily. The two modes in which GCA leads by a clear margin, `nli` and `bin`, are precisely the two that operate on the context and claim without splitting them. The two modes in which the advantage is negligible or negative, `bin_sp` and `nli_sp`, are the split variants, in which the evaluator itself segments the input and aggregates over the segments (Chapter 3, Section 3.3). Holistic accuracy also rises from 0.510–0.523 in the unsplit modes to 0.554–0.578 in the split modes, which is consistent with the split modes supplying a stronger holistic signal rather than with GCA becoming worse.

The pattern suggests that external sentence-level decomposition may contribute most where the evaluator does not already decompose internally, and little where it

does; on this reading GCA and the evaluator's own splitting act partly as substitutes. This sweep is exploratory and each mode was run once, so the comparison across modes should be read as suggestive rather than established: Section 6.6 shows that single-run differences of this magnitude fall within the range of run-to-run variation, and only the `nli` configuration was subsequently validated across repeated runs. Chapter 7, Section 7.2 develops the interpretation and states what would be required to test it.

Because a single run had already proved insufficient in Section 6.4, the `nli` result was rerun at the same seed value before any conclusion was drawn. The rerun gave holistic 0.543 and GCA 0.556, a gap of $+0.013$: smaller than the original $+0.060$, but in the same direction. Over the fold-level differences from both runs, the 95% interval was $[+0.003, +0.064]$ and the corresponding Wilcoxon p-value was 0.084. Under the independence caveat discussed above, neither figure was treated as decisive, and further runs were carried out rather than concluding from two.

6.6. Repeated Runs of the Selected Configuration ($n = 1{,}000$)

The `nli` mode, $\alpha = 0.0$, margin 0 configuration was fixed and run across four further seed values, in addition to the original sweep and its confirmation rerun, for six runs and 30 cross-validation folds in total. Table 6.5 lists every run. Note that the six runs span five distinct seed values, not six: seed 42 appears twice, as the original sweep and its rerun. As discussed in Chapter 4, Section 4.4, those two runs are not duplicates, because the seed argument controls only fold assignment and not the reward model's weight initialization.

Table 6.5.: Per-run results for the selected `nli`-mode configuration ($n = 1{,}000$).

Run	Seed	Holistic	GCA	Gap (GCA − Holistic)
Original sweep	42	0.523	0.583	$+0.060$
Confirmation	42	0.543	0.556	$+0.013$
Seed validation 1	7	0.556	0.546	-0.010
Seed validation 2	100	0.510	0.561	$+0.051$
Seed validation 3	314	0.520	0.552	$+0.032$
Seed validation 4	2026	0.525	0.564	$+0.039$

GCA leads in five of the six runs; seed 7 is the only run in which holistic is ahead. Figure 6.3 plots the per-run gaps against the pooled mean, which makes the spread easier to judge than the table alone: the individual runs range from -0.010 to $+0.060$, a spread wide enough that any single run, taken on its own, would have been weak evidence in either direction. Averaging over all 30 fold-level readings gives the central estimate of this thesis, shown in Table 6.6: a mean gap of $+3.08$ percentage

points.

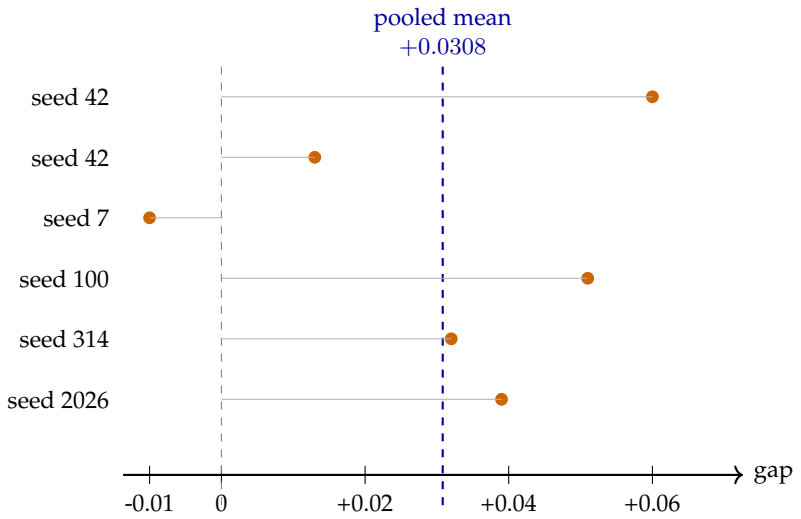


Figure 6.3.: Per-run GCA−Holistic accuracy gap at $n = 1{,}000$, against the pooled mean. Five of six runs favor GCA, but the spread across runs is wide relative to the pooled effect, which is why no single run was treated as conclusive.

Table 6.6.: Averaged result across 6 runs / 30 cross-validation folds at $n = 1{,}000$. The interval and p-value are computed over folds, which are not independent observations, and are therefore exploratory and anti-conservative (Chapter 4, Section 4.4).

Holistic mean	GCA mean	Gap	95% fold-level CI	Fold-level Wilcoxon p
0.5295	0.5603	**+0.0308**	$[+0.013, +0.047]$	0.0034

At the fold level, GCA leads in 22 of the 30 folds (73%), with no ties. The interval and p-value in Table 6.6 are computed as described in Chapter 3, Section 3.5, using 10,000 bootstrap resamples with a fixed NumPy seed of 42.

These uncertainty estimates require an explicit caveat and are not presented as confirmatory. The 30 folds are not 30 independent observations: the five folds within a run partition one dataset, so their training sets overlap heavily and their errors are correlated. A procedure that assumes independence therefore treats the evidence as stronger than it is, and the quoted p-value is anti-conservative. What the fold-level analysis supports is that the difference is consistent in direction across runs and stable in magnitude, not that it meets a particular significance threshold. Chapter 7, Section 7.3 examines what a more conservative treatment would and would not permit.

A second property should be read alongside the gap. Both conditions sit close to chance in absolute terms: 0.5295 and 0.5603 are 2.95 and 6.03 percentage points above the 0.5 chance level. The comparison is therefore between two weak preference signals, and the finding concerns which is more consistently recoverable, not whether either yields a strong reward model (Section 7.4).

Table 6.7 tracks how the estimate changed as runs were added. The interval narrows at each stage while the point estimate stays near three percentage points. Stability of the effect size as precision improves is consistent with a real underlying difference rather than with an artifact that grows as data accumulates, though the same independence caveat applies to every row of the table.

Table 6.7.: Progression of the pooled statistical estimate as runs were added. The p-values are subject to the independence caveat discussed in Chapter 7, Section 7.3.

Stage	Runs	Folds	Pooled gap	95% CI	Wilcoxon p
Two-run	2	10	$+0.034$	$[+0.003, +0.064]$	0.084
Five-run	5	25	$+0.029$	$[+0.009, +0.048]$	0.0123
Six-run	6	30	$+0.031$	$[+0.013, +0.047]$	0.0034

6.7. Scale Robustness: 5,000 and 10,000 Samples

The selected configuration was then run once at $n = 5{,}000$ and once at $n = 10{,}000$. These subsets are nested supersets of the $n = 1{,}000$ set rather than independent samples (Chapter 4, Section 4.6), so they measure how the effect behaves as data is added to the same pool rather than providing out-of-sample replication. Unlike the $n = 1{,}000$ result, each of these runs is a single run, not a six-run pool, for computational-budget reasons stated in Chapter 4, Section 4.4; the comparison in Table 6.8 should be read with that difference in mind rather than treated as three results of equal statistical weight.

Table 6.8.: Selected configuration at three dataset sizes. Only the $n = 1{,}000$ row is averaged over repeated runs; the other two are single runs on nested datasets.

Dataset size	Holistic mean	GCA mean	Gap (GCA − Holistic)
1,000	0.5295	0.5603	$+0.0308$
5,000	0.5788	0.5746	-0.0042
10,000	0.5827	0.5862	$+0.0035$

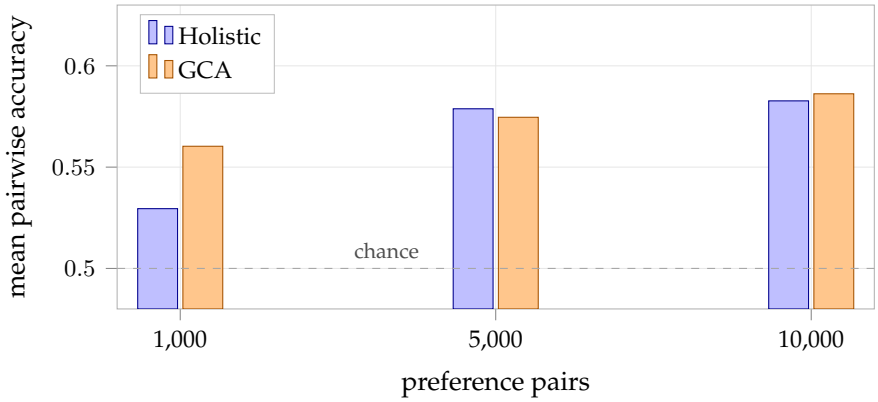


Figure 6.4.: Accuracy at the three dataset sizes. Absolute accuracy rises for both conditions as data is added, while the difference between them contracts. Only the $n = 1{,}000$ bars are averaged over repeated runs; the other two are single runs on datasets that are nested supersets of the $n = 1{,}000$ set (Chapter 4, Section 4.6).

The $n = 1{,}000$ advantage is not reproduced with comparable magnitude at $n = 5{,}000$: holistic performs marginally better there, though the gap is small. At $n = 10{,}000$ the direction returns in GCA's favor, but the magnitude is an order of magnitude smaller than at $n = 1{,}000$. Neither larger-scale result was repeated across runs the way the $n = 1{,}000$ result was, so it cannot be ruled out that some of this movement reflects the same run-to-run variation documented in Section 6.4 rather than a change in the underlying effect as dataset size grows. What can be said without qualification is that a single dataset size is not sufficient to characterize the difference between the two conditions, and that reporting only $n = 1{,}000$ would have overstated how consistently the GCA advantage is observed.

6.8. Summary of Results

Table 6.9 summarizes how the results in this chapter bear on the research questions from Chapter 4, Section 4.1. Full interpretation is left to Chapter 7.

Table 6.9.: Summary of findings against the research questions.

	Finding
RQ1	At $n = 1{,}000$, GCA leads in 5 of 6 runs with a mean difference of $+3.08$ percentage points. The run-level two-sided test does not meet $p < 0.05$, and the larger single-run experiments do not reproduce an advantage of comparable magnitude.
RQ2	Aggregation strategy and evaluator mode are associated with substantial outcome differences. The evaluator-mode comparison is exploratory, because every mode except the subsequently selected `nli` configuration was evaluated only once.
RQ3	The direction is relatively stable across the six $n = 1{,}000$ runs, but is not reproduced at comparable magnitude in the single $n = 5{,}000$ and $n = 10{,}000$ runs, which use nested datasets.

7. Discussion

7.1. What the Experiments Establish, and What They Do Not

The experiments support a narrow and conditional claim. Under one specific configuration, at one dataset size, preferences constructed by sentence-level aggregation are more consistently recoverable by a Bradley–Terry reward model than preferences constructed holistically from the same evaluator and the same candidate summaries. The difference averages roughly three percentage points over six repeated runs and is consistent in direction across five of them.

Four claims that might appear to follow do not. The results do not establish that GCA labels are factually more accurate, since no independent annotation was collected and AlignScore supplies both sets of labels (Section 7.7). They do not establish that a summarization model trained on GCA preferences produces more faithful summaries, since the final study contains no policy-optimization stage. They do not establish that GCA is generally preferable, since Chapter 6 reports configurations and scales at which it is not. And they do not establish a strong effect in absolute terms, since both reward models remain near chance (Section 7.4). What the study contributes is evidence about *when* feedback granularity changes the structure of a preference-learning signal, and the remainder of this chapter examines the conditions under which it does.

7.2. Why Granularity Interacts With Evaluator Design

The advantage reported in Chapter 6, Section 6.6 is not a property of GCA in the abstract. It appears only under a specific combination of two design choices, simple-mean aggregation ($\alpha = 0.0$) and the AlignScore `nli` evaluation mode, and Section 6.4 already showed that the aggregation-formula change alone is not sufficient: at $\alpha = 0.0$ under the `nli_sp` mode, holistic still led. This section asks why the judge mode specifically makes the difference.

AlignScore's implementation suggests a plausible mechanism, though the present experiments do not test it directly. The `_sp` suffix in `nli_sp` enables a preprocessing and aggregation step that is internal to AlignScore itself: the context is split into roughly 350-token chunks and the claim is split into sentences, and the final score is computed as, in the library's own `inference_per_example` function, the maximum entailment score across context chunks for each claim sentence, averaged across claim sentences (`.max(dim=0).values.mean()`). Plain `nli` mode skips this entirely and scores the full context against the full claim in one pass.

This has a direct consequence for what the holistic and GCA conditions actually compute under each mode. Under `nli_sp`, a holistic call already decomposes the candidate summary into sentences internally, scores each one against the best-matching chunk of the article, and averages the results, which is functionally close to what GCA does *externally* at $\alpha = 0.0$: score each summary sentence against the article and take the mean. When GCA is applied on top of a judge that is already doing something close to sentence-level decomposition and mean-pooling internally, its external decomposition adds little genuinely new information, which is consistent with holistic and GCA performing similarly, and even favoring holistic, under `nli_sp` (Table 6.4). Under plain `nli`, by contrast, a holistic call scores the entire summary against the (length-truncated) article as one atomic unit, with no internal sentence decomposition at all. Here, GCA's external per-sentence scoring is the only source of sentence-level localization in the pipeline, and it is exactly under this mode that GCA shows a measurable advantage.

The stored $n = 200$ preference data allow one component of this account to be checked directly rather than inferred. If a holistic `nli_sp` call internally splits the claim into sentences and averages, then the holistic score of a summary should closely track the mean of the separately computed per-sentence scores of that same summary. Across all 400 summaries in that set the two quantities correlate at $r = 0.983$, agree to within 0.001 for 234 of 400 summaries, and have a median absolute difference of 0.000 (Figure 7.1). Under `nli_sp`, holistic scoring and mean-aggregated sentence scoring are therefore close to the same computation, which is consistent with the observation that GCA at $\alpha = 0.0$ adds little under that mode.

This measurement supports the part of the account concerning `nli_sp`, but the account as a whole remains an interpretation of an exploratory sweep, and three limitations constrain it. The mode comparison rests on a single run per mode, and Section 7.8 shows that single-run differences of this size fall within observed run-to-run variation; only the selected `nli` configuration received repeated-run validation. No ablation isolates the components of the `_sp` preprocessing, such as chunk size, from the mode change as a whole. And nothing here verifies empirically that `nli_sp`'s internal aggregation and GCA's external aggregation produce comparable scores rather than merely comparable rankings. The proposition that evaluator-internal decomposition reduces the marginal benefit of external GCA is therefore offered as the most plausible reading of the observed pattern, and as a hypothesis worth testing directly, rather than as a finding of this thesis (Chapter 8).

7.3. How Much Weight the Statistical Evidence Can Carry

The $n = 1{,}000$ result is accompanied by a fold-level interval and a Wilcoxon p-value of 0.0034. Both are computed over 30 pooled fold-level differences, and it is necessary to state plainly how much they can support.

Those 30 differences are not 30 independent observations. Within a single run the five cross-validation folds partition one dataset: any two folds share four fifths

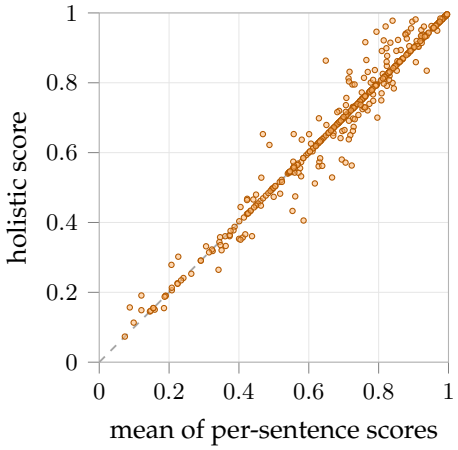


Figure 7.1.: Holistic AlignScore against the mean of the separately computed per-sentence scores, for all 400 candidate summaries in the $n = 200$ preference set under evaluator mode `nli_sp`. The dashed line is $y = x$. The two quantities are near-identical ($r = 0.983$), which is what the split mode's internal sentence-averaging predicts. The corresponding files for the final `nli` configuration are not retained in the repository, so the same check could not be run for the unsplit mode.

of their training data, and all five evaluate models trained from the same pool of preference pairs. Their errors are correlated, so a test assuming independence treats the evidence as stronger than it is. The effective sample size lies between the number of runs (six) and the number of folds (30), and closer to the former. The fold-level p-value is therefore anti-conservative and is reported here as an exploratory summary, not as a hypothesis test.

The appropriate unit of independent replication is the run. Applying the same two-sided Wilcoxon signed-rank test to the six run-level differences ($+0.060$, $+0.013$, -0.010, $+0.051$, $+0.032$, $+0.039$) gives $p = 0.0625$, which does not meet the conventional 0.05 threshold; the one-sided test in the direction favouring GCA gives $p = 0.0312$. With six observations, five of them positive, 0.0625 is in fact the smallest two-sided value the test can return, so this outcome reflects the number of runs available as much as the size of the effect. At the run level, then, the evidence does not meet the conventional threshold, and the thesis does not claim statistical significance for the $n = 1{,}000$ result.

Two properties of the result do not depend on any threshold. The difference is directionally consistent, favouring GCA in five of six runs that differ in both fold assignment and model initialization. And its magnitude, roughly three percentage points, is stable across pooling stages (Table 6.7) rather than drifting as runs were added. What the evidence supports is a directionally consistent, repeatedly observed advantage of approximately that size under this configuration, not a demonstrated

effect at a stated significance level.

7.4. Both Reward Models Are Weak in Absolute Terms

A comparison between two conditions can be sound while both conditions perform poorly, and that is the situation here. At $n = 1{,}000$, the holistic reward model reaches 0.5295 and the GCA reward model 0.5603 pairwise accuracy, against a chance level of 0.5. In absolute terms these models recover very little of the preference signal: the better of the two is correct on roughly 56 of every 100 held-out pairs, where guessing would be correct on 50.

This materially qualifies what the thesis shows. The finding is that GCA preferences are *more learnable* than holistic preferences under the selected configuration, not that either produces a reward model one would want to deploy. A reward model at 0.56 accuracy would provide a very noisy training signal to any downstream policy, and the repeatedly observed $+3.08$ percentage-point advantage is an improvement to a weak signal rather than the difference between a weak and a strong one.

Several factors plausibly contribute to the low absolute numbers, and this thesis can separate them only partially. The task itself is genuinely hard: both candidate summaries in a pair are generated by the same model from the same article and differ only in sampling temperature, so many pairs are near-ties in factual quality, and a judge's preference between them may encode little learnable structure. The RoBERTa-base reward model is also small, and articles are truncated to 2,000 characters before tokenization (Chapter 5, Section 5.5), so the model may simply not see enough of the source to verify many claims. The accuracies at $n = 5{,}000$ and $n = 10{,}000$ are higher for both conditions (0.58–0.59), which is consistent with more training data helping the reward model in general, even as the gap between conditions closes. Distinguishing these explanations would require ablations this thesis did not run, and the low absolute ceiling is recorded as a limitation in Chapter 8 rather than explained away.

7.5. Interpreting the Larger-Scale Results

The $n = 1{,}000$ advantage was not reproduced with comparable magnitude in the single runs at $n = 5{,}000$ and $n = 10{,}000$ (Chapter 6, Section 6.7). Three things must be kept apart here: the observed numerical behaviour, run-to-run variation, and hypotheses about dataset-size effects. The observation is that two single runs produced gaps of -0.42 and $+0.35$ percentage points where the pooled $n = 1{,}000$ estimate was $+3.08$. Whether this reflects a dataset-size effect is a separate question, and the design does not answer it.

The best-supported explanation is statistical rather than substantive. The $n = 1{,}000$ estimate itself only stabilized after six pooled runs; the individual per-run gaps in Table 6.5 ranged from -0.010 to $+0.060$, a spread wide enough that a single seed

could easily have landed on either side of zero. The $n = 5{,}000$ and $n = 10{,}000$ settings each contain a single run. Given how much the $n = 1{,}000$ estimate moved between its first two executions alone (from $+0.060$ down to $+0.013$), a single run at a different dataset size landing outside the $n = 1{,}000$ interval is not, by itself, strong evidence that the underlying effect changes with dataset size. It is at least as consistent with the same run-to-run variation already documented at $n = 1{,}000$, which simply went unmeasured at the larger sizes because only one run was performed at each.

A substantive explanation is also possible, but this thesis does not have evidence to support or rule it out: larger candidate pools could shift the distribution of article lengths, sentence counts, or segmentation quality in ways that change how much signal GCA's per-sentence scoring adds relative to holistic scoring. This is stated here as an open hypothesis, not a finding, since nothing in Chapter 6 isolates article length, sentence count, or segmentation quality as a variable.

Because these two explanations make different predictions, they are also distinguishable in principle: if the scale effect is genuinely statistical noise, repeated runs at $n = 5{,}000$ should produce an interval overlapping the $n = 1{,}000$ result; if it reflects a real, scale-dependent change, they should not. Repeating the $n = 5{,}000$ experiment across several runs is the most direct way to resolve this and is listed as future work in Chapter 8.

One hypothesis would predict this pattern, and it is worth stating even though the present data cannot test it. At $n = 1{,}000$ the reward model is fitting a small dataset, and how cleanly the preference relation is structured matters a great deal: a labeling that is internally more consistent is easier to recover from few examples. As the dataset grows, the model sees more examples of the same underlying relation, and additional data can compensate for a noisier labeling. Under that account, the benefit of a cleaner preference construction should be largest where data is scarce and should shrink as data accumulates, which is the pattern observed. Consistent with this, absolute accuracy rises for *both* conditions between $n = 1{,}000$ and $n = 10{,}000$ (from 0.5295 and 0.5603 to 0.5827 and 0.5862), while the difference between them contracts. The evidence is compatible with this explanation but does not establish it, since scale and dataset composition vary together across these runs.

Read this way, the larger-scale results are informative rather than merely disappointing. The observed pattern is consistent with the hypothesis that GCA may provide a larger learnability advantage when preference data are limited, which would also be the regime in which automatic preference construction is most attractive, since it is where collecting human labels would be most burdensome. The present design cannot establish such a boundary, because the larger-scale settings contain single runs on nested datasets. What the study does contribute here is the observation itself, together with a specification of the experiment that would test it.

7.6. Implications for RLAIF Preference Construction

Three implications follow for anyone constructing AI-generated preference data from a fixed factuality judge, independent of whether the specific GCA method studied here is adopted.

First, the aggregation formula used to combine sentence-level scores into a preference label is not a minor implementation detail. The original weakest-link-penalty formula ($\alpha = 0.5$) actively hurt GCA relative to a plain mean (Chapter 6, Table 6.2), which runs counter to the intuition that motivated it: a formula designed to be more sensitive to localized errors was, in practice, more sensitive to noise than to signal. Aggregation-formula choices of this kind should be validated empirically rather than assumed correct from first principles.

Second, judge configuration can matter more than the granularity decision that is nominally the point of comparison. Section 7.2's reading of AlignScore's mode behavior suggests that part of what looked like a question about holistic-versus-sentence-level supervision was really a question about how much sentence-level decomposition the judge was already doing internally. Anyone comparing supervision granularities on top of a judge with its own internal aggregation behavior should check what that judge already does before attributing an effect to the granularity of their own pipeline.

Third, a result validated at one dataset scale should not be treated as general without a scale check. This is a caution this thesis can make credibly only because Section 7.5's robustness check was actually run rather than assumed unnecessary; had the study stopped at the $n = 1{,}000$ result, it would have reported a materially less qualified conclusion than the evidence supports.

7.7. Judge Reliability Without Human Auditing

No independent human validation of the factuality labels was collected in this study, and it is worth stating precisely what this does and does not leave open. The repeated runs in Chapter 6, Section 6.6 establish that the difference between the two preference sets is repeatedly observed: it survives changes in fold assignment and training stochasticity, so it is not confined to a single favourable run.

Reproducibility and correctness are separate properties, however. That a preference relation is consistently recoverable says nothing about whether the underlying AlignScore judgments agree with what a human reader would consider faithful to the source. A judge can produce labels that are highly reproducible and systematically wrong, and nothing in this study would detect that case. The labels function here as the reference standard, not as verified ground truth, and every result is conditional on them in that sense. Establishing agreement with human factuality judgments requires annotation this study did not collect, and the question is recorded as an open limitation in Chapter 8.

7.8. Stochastic Variation Between Runs

Run-to-run variation is large enough in this setting to determine what conclusions can be drawn, and it deserves treatment as a finding rather than as an inconvenience. Two same-configuration runs differed by 4.7 percentage points ($+0.060$ versus $+0.013$), and across the six runs the gap ranged from -0.010 to $+0.060$. A study reporting only one of these runs could honestly have concluded either that GCA helps substantially or that it does not help at all.

This variation follows from an implementation property documented in Chapter 5, Section 5.5: the seed argument fixes the cross-validation fold assignment through Python's standard random number generator, but PyTorch's generator is not seeded in the cross-validation path. Several sources of training stochasticity are therefore uncontrolled, including reward-head weight initialization, the order in which training batches are drawn, and dropout masks. Runs are consequently not exactly reproducible from a seed value, which is a reproducibility weakness recorded in Chapter 8. It also has an interpretive consequence: the two executions at seed 42 are genuine repetitions rather than duplicates, because they share the fold assignment but not the training stochasticity. The spread between them reflects uncontrolled training stochasticity, including weight initialization, rather than initialization alone.

The practical implication generalizes beyond this study. At the accuracy levels and dataset sizes involved here, differences of one to two percentage points between conditions are within the range that a single run can produce by chance. Comparisons of preference-construction methods reported from single training runs at this scale should be treated with corresponding caution.

7.9. Confounds That Remain

Two confounds could not be removed within this design, and both plausibly affect the reported numbers.

The candidate pairs differ in decoding temperature, and Chapter 4, Table 4.3 shows the consequence is not small: the lower-temperature candidate is preferred in 68.5% of holistic pairs and 63.0% of GCA pairs on the data where this could be measured. Because temperature systematically affects surface properties such as length and lexical conservativeness, a reward model could recover a meaningful share of the preference relation from stylistic cues without representing factuality at all. The asymmetry also differs between conditions by roughly five percentage points, so the two preference sets are not equivalent with respect to this confound, and part of the observed difference between them could in principle reflect that rather than granularity. Sampling both candidates at the same temperature would remove the confound and is the cleanest correction available.

The reward model also sees only a truncated prefix of each source article (Chapter 5, Section 5.5), while subset selection admits articles several times longer than the retained prefix. Where the evidence bearing on a claim lies beyond that prefix, the

corresponding preference is not recoverable from the model's input even in principle. This is a plausible contributor to the low absolute accuracies discussed in Section 7.4, though its magnitude was not measured, and it should not be assumed to be the dominant cause without the ablation described there.

7.10. Generalizability

Every result reported here is obtained under one combination of choices: CNN/DailyMail as the corpus, AlignScore as the evaluator, Mistral-7B-Instruct-v0.3 as the candidate generator, and a RoBERTa-base Bradley–Terry model as the reward model. Each of these bounds the conclusions in a distinct way.

The evaluator is the most consequential. Section 7.2 raises the possibility that whether external decomposition helps is related to whether the evaluator already decomposes internally, which would make the finding partly *about* the evaluator's architecture rather than simply conditional on AlignScore as a fixed instrument. If that reading is correct, a different factuality model with different internal aggregation would shift the result. The corpus matters because CNN/DailyMail summaries are short, extractive-leaning, and drawn from a single register; domains with longer or more abstractive summaries would change both the number of sentences per summary and the difficulty of the underlying factuality judgments. The reward-model backbone matters because a stronger encoder, or one with a longer context window, might recover more of the preference relation and thereby narrow or widen the difference between conditions.

Taken together, these dependencies mean the study is best read as characterizing behaviour under one specific configuration, and as motivating a hypothesis about the interaction between external and evaluator-internal decomposition, rather than as establishing a quantity that transfers directly to other setups.

8. Conclusion

8.1. Summary

This thesis asked a controlled question: does converting sentence-level factuality scores into a summary-level preference through Granular Credit Assignment produce a more learnable reward-model signal than scoring each candidate summary holistically. The two conditions were compared under an identical candidate pool, an identical factuality judge, and an identical Bradley–Terry training recipe, so that the only manipulated factor was the preference-construction procedure itself.

Averaged over six repeated runs at $n = 1{,}000$, GCA preferences yield an advantage of approximately $+3.08$ percentage points in reward-model pairwise accuracy. The advantage does not remain equally large as the dataset grows: it is approximately -0.42 percentage points at $n = 5{,}000$ and $+0.35$ percentage points at $n = 10{,}000$, while absolute accuracy improves for both conditions. Uncertainty estimates computed over cross-validation folds are anti-conservative, because folds within a run are not independent, so the direction and approximate magnitude of the effect are better supported than any exact significance level.

The experiments therefore do not support a universal claim that GCA is preferable to holistic factuality supervision. They show that sentence-level aggregation can improve the learnability of automatically generated pairwise preferences under particular configurations, most clearly in the $n = 1{,}000$ setting. In the exploratory mode comparison, the advantage appears under evaluator modes that do not themselves decompose the input, which motivates the hypothesis that external and evaluator-internal decomposition may act partly as substitutes. At larger sample sizes the observed advantage is small and inconsistent in sign. Taken together, the results indicate that the usefulness of granular credit assignment varies with evaluator configuration, dataset size, and aggregation choice rather than following from finer granularity as such.

8.2. Answers to the Research Questions

RQ1 (does GCA produce a more learnable preference signal than holistic scoring?) is answered *conditionally*. An average advantage of approximately 3 percentage points is repeatedly observed at $n = 1{,}000$, favouring GCA in five of six runs, though the run-level test does not meet the conventional threshold. It is not observed in the single run at $n = 5{,}000$ and is marginal at $n = 10{,}000$. The answer is therefore configuration-dependent rather than general.

RQ2 (how do evaluator configuration and aggregation strategy affect relative learnability?) is answered *associationally*. Both the aggregation formula and the evaluator mode are associated with substantial changes in the outcome, and in the exploratory sweep the two modes in which the evaluator performs its own internal decomposition are also the two in which GCA shows no advantage (Chapter 6, Section 6.5). Because only one mode received repeated-run validation, this pattern is suggestive rather than established.

RQ3 (how stable are the differences across runs and dataset sizes?) is answered *partially*. The difference is directionally consistent across six runs at fixed $n = 1{,}000$, favouring GCA in five of six. It is not reproduced with comparable magnitude in the single runs at $n = 5{,}000$ and $n = 10{,}000$, which use nested supersets of the same data (Section 7.5).

8.3. Limitations

1. **No human factuality audit.** No independent human annotation was collected in the final study, so the evaluator's judgments were never checked against human assessments of faithfulness (Chapter 7, Section 7.7).
2. **The labels are not independent ground truth.** AlignScore supplies both the holistic and the sentence-level scores, so it functions as the reference standard rather than as a verified one. Every result is conditional on that evaluator.
3. **Preference learnability is not factual correctness.** The dependent variable measures how consistently a preference relation can be recovered by the chosen architecture, not whether the preferences are factually right (Chapter 4, Section 4.3).
4. **No downstream policy evaluation.** The final study contains no DPO or other policy-optimization stage, so it does not establish whether models trained on either preference set generate more factually consistent summaries.
5. **Both reward models are weak in absolute terms.** At $n = 1{,}000$ the conditions reach 0.5295 and 0.5603 pairwise accuracy against a chance level of 0.5, so the comparison is between two weak signals (Chapter 7, Section 7.4).
6. **Cross-validation folds are not independent.** Uncertainty estimates computed over 30 folds treat correlated observations as independent and are therefore anti-conservative; the direction and approximate magnitude of the effect are better supported than any exact significance level (Section 7.3).
7. **Incomplete determinism across runs.** Subset selection and cross-validation fold assignment are explicitly seeded, but PyTorch's generator is not seeded in the cross-validation path, leaving weight initialization, batch ordering and dropout uncontrolled (Chapter 5, Section 5.5). Exact run-level reproduction from the command-line seed alone is therefore not guaranteed.

8. **Configuration selected on observed results.** The aggregation strategy and evaluator mode were chosen after inspecting $n = 1{,}000$ outcomes (Chapter 4, Section 4.5), and the larger-scale subsets are nested supersets of that same data (Section 4.6), so no result in this thesis evaluates the selected configuration on independent data.

9. **Candidate generation confounds temperature with condition.** The two candidates differ in decoding temperature, and the lower-temperature candidate is preferred substantially more often under both conditions and to differing degrees (Chapter 4, Section 4.8), so temperature-correlated features may carry part of the label information.

10. **Source-document truncation.** The reward model receives only a truncated prefix of each article, so evidence located later in the source is unavailable to it (Chapter 5, Section 5.5).

11. **Evaluator and dataset specificity.** All findings are obtained on CNN/DailyMail with AlignScore, Mistral-7B-Instruct-v0.3 candidates, and a RoBERTa-base reward model, and need not transfer to other domains, evaluators, or architectures.

12. **Sentence segmentation was not evaluated.** Summaries are split with a regex-based splitter whose accuracy was never measured (Chapter 5, Section 5.4).

13. **Nested dataset sizes.** The $n = 1{,}000$, $n = 5{,}000$ and $n = 10{,}000$ subsets overlap almost completely (Chapter 4, Section 4.6), so the larger-scale runs are not independent samples and no experiment evaluates the selected configuration on held-out data.

14. **No error-category analysis.** No analysis links the observed effects to specific categories of factual error; this was not part of the final research questions and is listed under future work.

8.4. Future Work

- **Repeated runs at larger scale.** The $n = 5{,}000$ and $n = 10{,}000$ experiments were each executed once. Repeating them would test whether their intervals overlap the $n = 1{,}000$ estimate, which would indicate run-to-run variance, or remain separated, which would support a dataset-size effect (Chapter 7, Section 7.5).

- **Remove the temperature confound.** Generating both candidates at a single decoding temperature, and comparing the resulting preference sets against the current ones, would establish how much of the observed structure is attributable to temperature-correlated surface features rather than to factuality (Section 7.9).

- **Measure the effect of source truncation.** Varying the retained article length while holding all else fixed would quantify how much of the near-chance reward-model accuracy is attributable to evidence falling outside the model's input window (Section 7.4).
- **Test the decomposition-interaction mechanism directly.** Section 7.2 infers from AlignScore's documented behavior that its split modes perform internally much of what GCA performs externally. Comparing per-sentence scores under split and unsplit modes on the same pairs would test that inference rather than rest on it.
- **Independent validation of the labels.** Human annotation of a stratified sample of holistic/GCA disagreement cases, or scoring the same candidate pool with a second factuality model, would provide the external reference this study lacks and would indicate whether either construction procedure produces labels closer to human judgment.
- **Downstream policy evaluation.** Training a summarization policy on each preference set and evaluating the factual consistency of its generations would test whether differences in preference learnability translate into differences in generated output, which this study deliberately does not address (Chapter 4, Section 4.7).
- **Error-category analysis.** Categorizing disagreement cases by error type would test whether any advantage concentrates in localized errors such as entity or relation mistakes, as originally hypothesized.

Bibliography

[1] Yuntao Bai et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

[2] Stephen Casper et al. Open problems and fundamental limitations of reinforcement learning from human feedback. *arXiv preprint arXiv:2307.15217*, 2023.

[3] Paul Christiano et al. Deep reinforcement learning from human preferences. *arXiv preprint arXiv:1706.03741*, 2017.

[4] Esin Durmus, He He, and Mona Diab. FEQA: A question answering evaluation framework for faithfulness assessment in abstractive summarization. In *Proceedings of ACL*, 2020.

[5] Alexander R. Fabbri et al. QAFactEval: Improved QA-based factual consistency evaluation for summarization. In *Proceedings of NAACL*, 2022.

[6] Jiawei Gu et al. A survey on LLM-as-a-judge. *arXiv preprint arXiv:2411.15594*, 2024.

[7] Yuzhe Gu, Wenwei Zhang, Chengqi Lyu, Dahua Lin, and Kai Chen. Mask-DPO: Generalizable fine-grained factuality alignment of LLMs. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2503.02846.

[8] Karl Moritz Hermann et al. Teaching machines to read and comprehend. *arXiv preprint arXiv:1506.03340*, 2015.

[9] Wojciech Kryściński et al. Evaluating the factual consistency of abstractive text summarization. In *Proceedings of EMNLP*, 2020.

[10] Philippe Laban et al. SummaC: Re-visiting NLI-based models for inconsistency detection in summarization. *Transactions of the Association for Computational Linguistics*, 2022.

[11] Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, Noah A. Smith, and Hannaneh Hajishirzi. RewardBench: Evaluating reward models for language modeling. *arXiv preprint arXiv:2403.13787*, 2024.

[12] Harrison Lee et al. RLAIF vs. RLHF: Scaling reinforcement learning from human feedback with AI feedback. *arXiv preprint arXiv:2309.00267*, 2023.

[13] Yang Liu et al. G-eval: NLG evaluation using GPT-4 with better human alignment. *arXiv preprint arXiv:2303.16634*, 2023.

[14] Ramesh Nallapati et al. Abstractive text summarization using sequence-to-sequence RNNs and beyond. *arXiv preprint arXiv:1602.06023*, 2016.

[15] Long Ouyang et al. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.

[16] Wenjie Qiu, Yi-Chen Li, Xuqin Zhang, Tianyi Zhang, Yihang Zhang, Zongzhang Zhang, and Yang Yu. Sentence-level reward model can generalize better for aligning LLM from human preference. *arXiv preprint arXiv:2503.04793*, 2025. doi: 10.48550/arXiv.2503.04793.

[17] Rafael Rafailov et al. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023.

[18] John Schulman et al. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[19] Abigail See, Peter J. Liu, and Christopher D. Manning. Get to the point: Summarization with pointer-generator networks. *arXiv preprint arXiv:1704.04368*, 2017.

[20] Hwanjun Song, Hang Su, Igor Shalyminov, Jason Cai, and Saab Mansour. FineSurE: Fine-grained summarization evaluation using LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 906–922, 2024. doi: 10.18653/v1/2024.acl-long.51.

[21] Nisan Stiennon et al. Learning to summarize from human feedback. *arXiv preprint arXiv:2009.01325*, 2020.

[22] Zeqiu Wu et al. Fine-grained human feedback gives better rewards for language model training. *arXiv preprint arXiv:2306.01693*, 2023.

[23] Yuheng Zha, Yichi Yang, Ruichen Li, and Zhiting Hu. AlignScore: Evaluating factual consistency with a unified alignment function. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.

[24] Lianmin Zheng et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023.

A. Data and Code Availability

All source code, configuration files, Slurm job scripts, and analysis notebooks used to produce the results reported in this thesis are publicly available at:

`https://github.com/hasnat23/master-thesis-rlaif-gca`

The repository is organized as follows. `src/` contains the pipeline implementation, split into the modules referenced throughout Chapter 5: `src/data/` (subset selection and record schemas), `src/generation/` (candidate summary generation), `src/judging/` (the AlignScore judge wrapper and both preference-construction regimes), `src/reward_model/` (Bradley–Terry training and cross-validation), and `src/eval/` and `src/analysis/` (evaluation and statistics). `configs/` holds the YAML configuration files for subset selection, generation, and judging. `slurm/` holds the batch scripts submitted on MOGON NHR, including `slurm/train_rm_1000.sh`, which produced the headline $n = 1{,}000$ result. `analysis/` holds the standalone scripts used for the formula sweep, the judge-mode sweep, and the holistic-versus-GCA comparison. `progress-updates/` contains the dated working notes kept over the course of the project.

The underlying corpus, CNN/DailyMail version `3.0.0` [8, 14], is publicly available through the Hugging Face `datasets` library and is not redistributed in the repository. The candidate summaries, preference files, and trained reward-model checkpoints generated during this project are likewise not committed, as they run to several gigabytes; they can be regenerated from the corpus using the configuration recorded in Appendix B. The factuality evaluator, `yzha/AlignScore` [23], and the candidate generator, Mistral-7B-Instruct-v0.3, are both publicly released model checkpoints obtained from the Hugging Face Hub.

B. Reproduction Configuration

This appendix records the exact configuration required to reproduce the headline $n = 1{,}000$ result. It restates in executable form what Chapter 4, Table 4.1 summarizes and Chapter 5 describes in prose. The reproducibility caveat stated in Chapter 5, Section 5.7 applies: because PyTorch's generator is not seeded in the cross-validation path, two executions with identical arguments can differ, and these commands reproduce the experimental *setup* rather than guaranteeing bit-identical numbers.

B.1. Summarization Prompt

Both candidate summaries for a given article are generated from the same fixed prompt template (`SUMMARIZATION_PROMPT` in `src/generation/candidates.py`); the two candidates differ only in the sampling parameters applied at decoding time.

```
Summarize the following news article in a concise paragraph.

Article:
{article}

Summary:
```

B.2. Pipeline Parameters

Table B.1 lists the parameters of each pipeline stage as used for the final comparison. Where a value differs from the default recorded in the corresponding YAML file, the command-line override in Section B.3 is authoritative; this is the case for the subset size, the tie margin, and the GCA exponent α, each of which changed over the course of the project as described in Chapter 4, Section 4.7.

B.3. Command-Line Invocations

The three stages are run in sequence. Preference construction produces both conditions from a single invocation, and reward-model training consumes both preference files so that the holistic and GCA models are trained under an identical recipe within one process.

```
# 1. Subset selection
```

Table B.1.: Pipeline parameters for the final $n = 1{,}000$ comparison.

Stage	Parameter	Value
Subset selection	Corpus	CNN/DailyMail 3.0.0, `test` split
	Samples	1,000
	Subset seed	200
	Max article length	8,000 characters
	Max summary length	2,000 characters
Candidate generation	Model	Mistral-7B-Instruct-v0.3, `bfloat16`
	Candidate A	$T = 0.7$, top-$p = 0.9$
	Candidate B	$T = 1.0$, top-$p = 0.95$
	Max input tokens	1,024
	Max new tokens	256
	Batch size	4
Preference construction	Evaluator	`yzha/AlignScore`, `FacebookAI/roberta-base nli`
	Evaluation mode	
	GCA exponent	$\alpha = 0.0$ (simple mean)
	Tie margin	0
Reward-model training	Backbone	`FacebookAI/roberta-base`
	Epochs	5
	Batch size	8
	Learning rate	2×10^{-5}, linear warmup over 10% of steps
	Max sequence length	512 tokens
	Max article chars	2,000
	Cross-validation	5-fold; training seeds 42, 42, 7, 100, 314, 2026

```
python scripts/01_prepare_subset.py --n-samples 1000 --seed 200

# 2. Candidate generation
python scripts/02_generate_candidates.py \
    --config configs/generation_1000.yaml

# 3. Preference construction (both conditions)
python src/judging/build_reward_preferences.py \
    --mode both --alpha 0.0 --margin 0 \
    --alignscore-evaluation-mode nli \
    --alignscore-backbone FacebookAI/roberta-base

# 4. Reward-model training (5-fold CV, both conditions)
python src/reward_model/run_training.py \
    --holistic data/preferences_1000/\
holistic_reward_preferences_1000.jsonl \
    --gca      data/preferences_1000/\
gca_reward_preferences_1000.jsonl \
    --output-dir outputs/reward_models_1000 \
    --backbone FacebookAI/roberta-base \
    --epochs 5 --batch-size 8 --lr 2e-5 \
    --max-length 512 --max-article-chars 2000 \
    --kfold 5 --seed 42
```

In step 4 the two preference-file paths are shown split across lines for typesetting only; the trailing backslash inside a path is a line continuation in this listing, not part of the filename.

On the MOGON NHR cluster these are submitted as Slurm batch jobs (`slurm/generate_candidates_1000.sh`, `slurm/build_preferences_1000.sh`, `slurm/train_rm_1000.sh`) on partition `a100dl` with one A100-SXM4-40GB GPU, 32 GB of host memory, and four CPU cores per task.

C. Example Preference Record

Every record written by the GCA preference-construction stage retains the full sentence-level detail behind the summary-level decision, which is what makes the qualitative disagreement analysis in Chapter 6 possible. The listing below shows one such record, abridged: the article, reference summary, and the `chosen`/`rejected` fields are truncated, and the sentence list for summary B is cut after the first three sentences. All numeric values are reproduced verbatim.

The record shown was produced by the earlier $n = 200$ run, which used the exploratory settings $\alpha = 0.5$ and margin $= 0.05$ rather than the final $\alpha = 0.0$ and margin $= 0$ (Chapter 4, Section 4.7). It is included to illustrate the record *schema* and the relationship between the sentence scores and the aggregate, not as an instance of the final configuration. Note in particular that `gca_score_a` $= 0.382$ is well below `mean_score_a` $= 0.629$: this is the penalty term at $\alpha = 0.5$ pulling the aggregate down towards the summary's weakest sentence, the behaviour that Chapter 6, Section 6.3 found to perform worse than the simple mean.


```
{
  "sample_id": "cnn_00000_b0b34b2d843e",
  "article": "LONDON, England (Reuters) -- Harry Potter star Daniel ...",
  "reference_summary": "Harry Potter star Daniel Radcliffe gets ...",
  "summary_a": "Daniel Radcliffe, the 18-year-old actor who plays ...",
  "summary_b": "On Monday, July 23, 2007, Harry Potter actor Daniel ...",
  "gca_score_a": 0.382102,
  "gca_score_b": 0.169184,
  "gca_score_diff": 0.212918,
  "mean_score_a": 0.629039,
  "mean_score_b": 0.578186,
  "min_score_a": 0.232103,
  "min_score_b": 0.049505,
  "n_sentences_a": 3,
  "n_sentences_b": 6,
  "n_low_faithfulness_a": 1,
  "n_low_faithfulness_b": 3,
  "decision": "A",
  "judge_method": "gca",
  "judge_model": "yzha/AlignScore [FacebookAI/roberta-base]",
  "margin": 0.05,
  "alpha": 0.5,
  "sentence_details_a": [
    { "index": 0,
      "text": "Daniel Radcliffe, the 18-year-old actor who plays Harry
               Potter, has gained access to a 20 million pound fortune
               on his 18th birthday.",
```

```
      "score": 0.748840, "label": "faithful" },
    { "index": 1,
      "text": "Despite the newfound wealth, he has stated that he does
               not plan to be extravagant, and will instead use his
               money to buy books, CDs, and DVDs.",
      "score": 0.232103, "label": "hallucinated" },
    { "index": 2,
      "text": "Radcliffe has filmed a TV movie and an Australian film,
               and has made his stage debut in \"Equus.\" He is expected
               to face closer media scrutiny now that he is legally an
               adult.",
      "score": 0.906173, "label": "faithful" }
  ],
  "sentence_details_b": [
    { "index": 0, "text": "On Monday, July 23, 2007, ...",
      "score": 0.475803, "label": "hallucinated" },
    { "index": 1, "text": "Radcliffe has stated he has no ...",
      "score": 0.997014, "label": "faithful" },
    { "index": 2, "text": "Instead, he plans to use the money ...",
      "score": 0.049505, "label": "hallucinated" }
  ]
}
```


Two features of this record are worth drawing out, because they illustrate mechanisms discussed in the main text. First, the `label` field is a threshold applied at a score of 0.5 for reporting convenience only; it plays no part in the aggregation, which operates on the continuous scores. Second, summary B contains both the highest-scoring sentence in the pair (0.997) and the lowest (0.050). Under the holistic condition this internal variation is not observable at all: the summary receives a single score. This is precisely the information that sentence-level decomposition exposes and holistic scoring discards, and whether exposing it yields a more learnable preference signal is the question the thesis sets out to answer.